

Тема 5.2 Основы фреймворка Django

Создание и использование шаблонов

Шаблоны (templates) отвечают за формирование внешнего вида приложения. Они предоставляют специальный синтаксис, который позволяет внедрять данные в код HTML.

Допустим, у нас есть проект metanit, и в нем определено одно приложение – hello (рисунок 1):

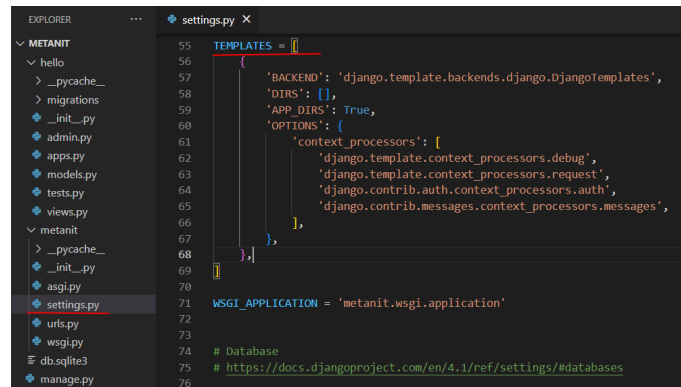


Рисунок 1 - Шаблоны Templates в Django

Настройка функциональности шаблонов в проекте Django производится в файле settings.py. с помощью переменной TEMPLATES. Так, по умолчанию переменная TEMPLATES в файле settings.py имеет следующее определение:

```
TEMPLATES = [  
    {  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
        'DIRS': [],  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    },  
]
```

Данная переменная принимает список конфигураций для каждого движка шаблонов. По умолчанию определена одна конфигурация, которая имеет следующие параметры:

- BACKEND: движок шаблонов. По умолчанию применяется встроенный движок django.template.backends.django.DjangoTemplates;
- DIRS: определяет список каталогов, где движок шаблонов будет искать файлы шаблонов. По умолчанию пустой список;

- APP_DIRS: указывает, будет ли движок шаблонов искать шаблоны внутри папок приложений в папке templates;
- OPTIONS: определяет дополнительный список параметров

Итак, в конфигурации по умолчанию параметр APP_DIRS имеет значение True, а это значит, что движок шаблонов будет также искать нужные файлы шаблонов в папке приложения в каталоге templates. То есть по умолчанию мы уже имеем настроенную конфигурацию, готовую к использованию шаблонов. Теперь определим сами шаблоны.

Добавим в папку приложения каталог templates. А в нем определим файл index.html (рисунок 2):

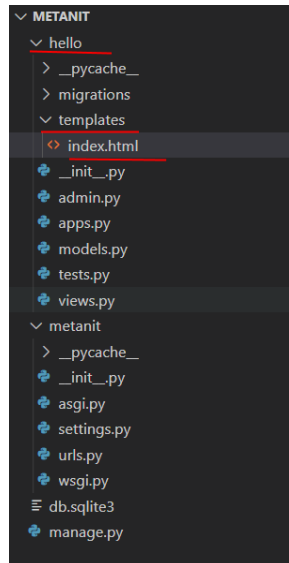


Рисунок 2 - Добавление шаблонов Templates в проект на Django и Python

Далее в файле index.html определим следующий код:

```
<!DOCTYPE html>
<html>
<head>
  <title>Django на METANIT.COM</title>
  <meta charset="utf-8" />
</head>
<body>
  <h2>Hello METANIT.COM</h2>
</body>
</html>
```

По сути, это обычная веб-страница, которая содержит код html. Теперь используем эту страницу для отправки ответа пользователю. И для этого перейдем в приложении hello к файлу views.py, который определяет функции для обработки запроса. Изменим этот файл следующим образом:

```
from django.shortcuts import render
def index(request):
    return render(request, "index.html")
```

Из модуля django.shortcuts импортируется функция render.

Функция `index` вызывает функцию `render`, которой передаются объект запроса `request` и путь к файлу шаблона в рамках папки `templates` - `"index.html"`.

В файле `urls.py` проекта пропишем сопоставление функции `index` с запросом к корню веб-приложения (рисунок 3):

```
from django.urls import path
from hello import views
urlpatterns = [
    path("", views.index),
]
```

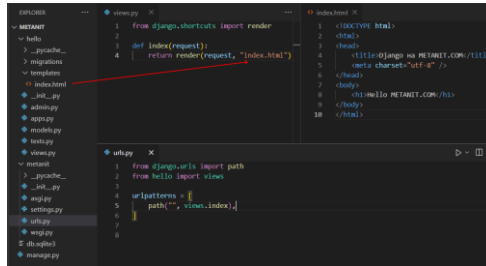


Рисунок 3 - Связь шаблона `template` и `view` в Django

И запустим проект на выполнение и перейдем к приложению в браузере (если проект запущен, то его надо перезапустить) (рисунок 4):

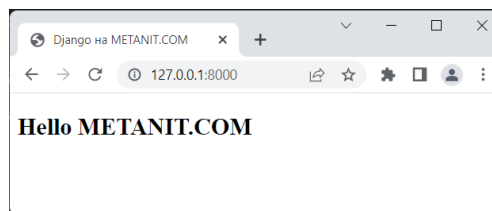


Рисунок 4 - Шаблоны в веб-приложении на Django и python

Подобным образом можно указать и другие шаблоны. Например, в папку `templates` добавим еще две страницы: `about.html` и `contact.html` (рисунок 5).

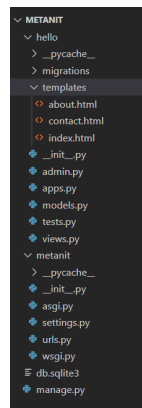


Рисунок 5 - Шаблоны в Python и Django

И также в файле `views.py` определим функции, которые используют данные шаблоны:

```
from django.shortcuts import render
def index(request):
    return render(request, "index.html")
def about(request):
    return render(request, "about.html")
def contact(request):
    return render(request, "contact.html")
```

А в файле `urls.py` свяжем функции с маршрутами:

```
from django.urls import path
from hello import views
urlpatterns = [
    path("", views.index),
    path("about/", views.about),
    path("contact/", views.contact),
]
```

Выше для генерации шаблона применялась функция `render()`, которая является наиболее распространенным вариантом. Однако также мы можем использовать класс `TemplateResponse`:

```
from django.template.response import TemplateResponse
def index(request):
    return TemplateResponse(request, "index.html")
```

Результат будет тот же самый.

Обработка запроса

Центральным моментом любого веб-приложения является обработка запроса, который отправляет пользователь. В Django за обработку запроса отвечают представления или `views`. По сути представления представляют функции обработки, которые принимают данные запроса в виде объекта `HttpRequest` из пакета `django.http` и генерируют некоторый результат, который затем отправляется пользователю.

По умолчанию представления размещаются в приложении в файле `views.py`.

Например, возьмем стандартный проект, в который добавлено приложение (рисунок 6).

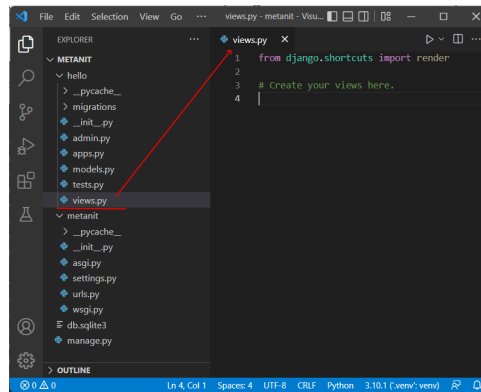


Рисунок 6 - Представления Views в приложении на Django и Python

При создании нового проекта файл `views.py` имеет следующее содержимое:

```
from django.shortcuts import render
# Create your views here.
```

Данный код пока никак не обрабатывает запросы, он только импортирует функцию `render()`, которая может использоваться для обработки.

Генерировать результат можно различными способами. Один из них представляет использование класса `HttpResponse` из пакета `django.http`, который позволяет отправить текстовое содержимое.

Так, изменим файл `views.py` следующим образом:

```
from django.http import HttpResponse
def index(request):
    return HttpResponse("Главная")
def about(request):
    return HttpResponse("О сайте")
def contact(request):
    return HttpResponse("Контакты")
```

В данном случае определены три функции, которые будут обрабатывать запросы. Каждая функция принимает в качестве параметра `request` объект `HttpRequest`, который хранит информацию о запросе. Однако в данном случае она нам не нужны, поэтому параметр никак не используется. Для генерации ответа в конструктор объекта `HttpResponse` передается некоторая строка. Это может быть в том числе и код `html` в виде строки.

Чтобы эти функции сопоставлялись с запросами, надо определить для них маршруты в проекте в файле `urls.py`. В частности, изменим этот файл следующим образом (рисунок 7):

```
from django.urls import path
from hello import views
urlpatterns = [
    path("", views.index),
    path('about', views.about),
    path('contact', views.contact),
]
```

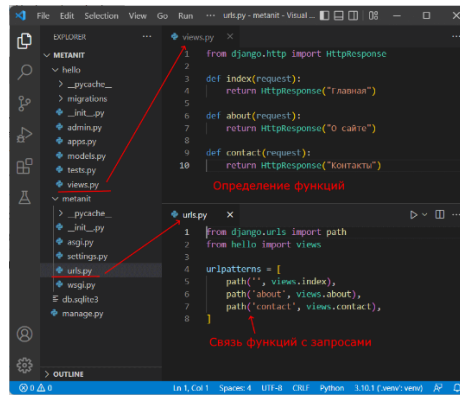


Рисунок 7 - Обработка запроса в Django и Python

Переменная `urlpatterns` определяет набор сопоставлений функций обработки с определенными строками запроса. Например, запрос к корню веб-сайта будет обрабатываться функцией `index`, запрос по адресу "about" будет обрабатываться функцией `about`, а запрос "contact" - функцией `contact`.

Запустим проект и обратимся по некоторым из этих адресов (рисунок 8).

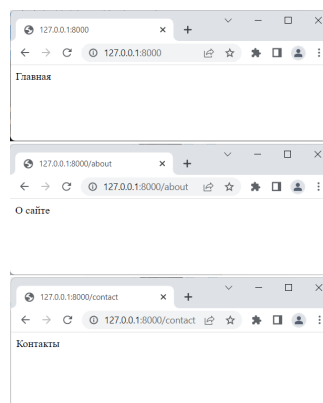


Рисунок 8 - Сопоставление запросов с функциями в Django и Python

При этом мы можем отправлять не простой текст, а, например, код html, который затем интерпретируется браузером. Так, изменим файл `views.py` следующим образом:

```
from django.http import HttpResponse
def index(request):
    return HttpResponse("<h2>Главная</h2>")
def about(request):
    return HttpResponse("<h2>О сайте</h2>")
def contact(request):
    return HttpResponse("<h2>Контакты</h2>")
```

Соответственно теперь браузер получит код html (рисунок 9):

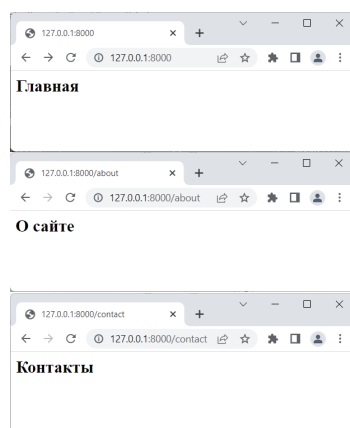


Рисунок 9 - Обработка запросов и HttpResponseRedirect в Django и Python

Практическая часть

Центральным моментом любого веб-приложения является обработка запроса, который отправляет пользователь. В Django за обработку запроса отвечают представления или **views**. По сути представления представляют функции обработки, которые принимают данные запроса в виде объекта **HttpRequest** из пакета **django.http** и генерируют некоторый результат, который затем отправляется пользователю.

Обработка запроса – процесс принятия и обработки данных запроса пользователя в веб-приложении.

По умолчанию представления размещаются в приложении в файле **views.py**. Например, возьмем стандартный проект, в который добавлено приложение (рисунок 10).

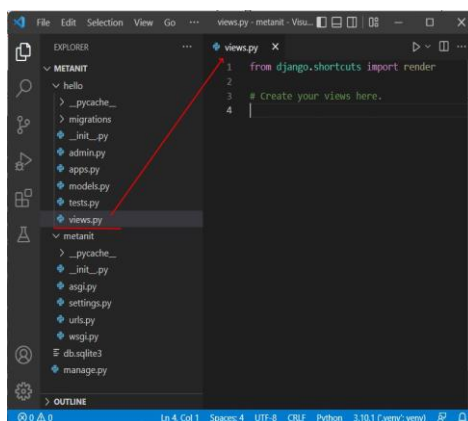


Рисунок 10 - Файл views.py

При создании нового проекта файл **views.py** имеет следующее содержимое (рисунок 11):

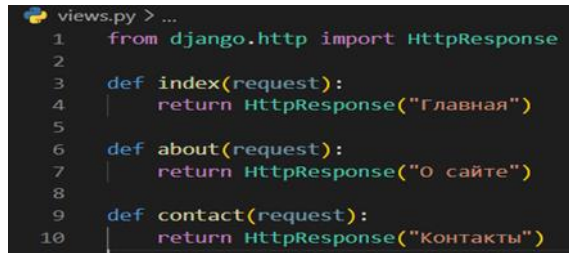
```
1  from django.shortcuts import render
2
3  # Create your views here.
```

Рисунок 11 - Базовый вид файла views.py

Данный код пока никак не обрабатывает запросы, он только импортирует функцию **render()**, которая может использоваться для обработки.

Генерировать результат можно различными способами. Один из них представляет использование класса **HttpResponse** из пакета `django.http`, который позволяет отправить текстовое содержимое.

Так, изменим файл **views.py** следующим образом (рисунок 12):



```

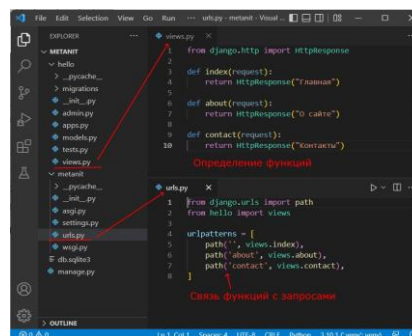
1  from django.http import HttpResponse
2
3  def index(request):
4      return HttpResponse("Главная")
5
6  def about(request):
7      return HttpResponse("О сайте")
8
9  def contact(request):
10     return HttpResponse("Контакты")

```

Рисунок 12 - Измененный файл с функциями для отображения главной страницы, страницы о сайте и контактов

В данном случае определены три функции, которые будут обрабатывать запросы. Каждая функция принимает в качестве параметра `request` объект **HttpRequest**, который хранит информацию о запросе. Однако в данном случае она нам не нужны, поэтому параметр никак не используется. Для генерации ответа в конструктор объекта `HttpResponse` передается некоторая строка. Это может быть в том числе и код `html` в виде строки.

Чтобы эти функции сопоставлялись с запросами, надо определить для них маршруты в проекте в файле **urls.py**. В частности, изменим этот файл следующим образом (рисунок 13):



```

1  from django.urls import path
2  from hello import views
3
4  urlpatterns = [
5      path('', views.index),
6      path('about', views.about),
7      path('contact', views.contact),
8  ]

```

Рисунок 13 - Связь между файлами views.py и urls.py

Переменная **urlpatterns** определяет набор сопоставлений функций обработки с определенными строками запроса. Например, запрос к корню веб-сайта будет обрабатываться функцией `index`, запрос по адресу "about" будет обрабатываться функцией `about`, а запрос "contact" - функцией `contact`.

Запустим проект и обратимся по некоторым из этих адресов (рисунок 14).



Рисунок 14 - Отображение наших представлений в браузере

Вернемся к нашему проекту и перейдем в файл `views.py`. У уже известной функции `render` – есть такой параметр, как `context`, он и позволяет нам выводить определенные данные на сайте (если логика прописана правильно) (рисунок 15).

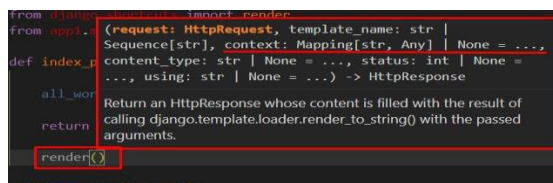


Рисунок 15 - Подсказка в Visual Studio Code о параметрах функций

Передадим в данный `context` словарь (пара «ключ-значение») какие-нибудь данные, например (рисунок 16):

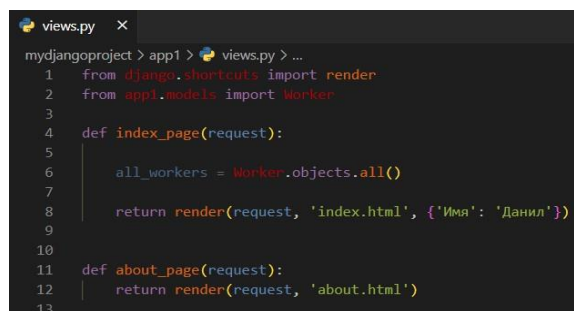
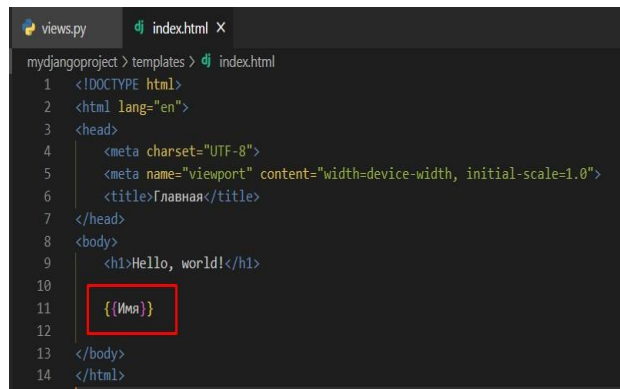


Рисунок 16 - Передача данных в context

Перейдем в `index.html` и укажем в двойных фигурных скобках значение КЛЮЧА словаря (рисунок 17).



```

views.py  index.html X
mydjango project > templates > index.html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Главная</title>
7  </head>
8  <body>
9      <h1>Hello, world!</h1>
10
11     {{Имя}}
12
13 </body>
14 </html>

```

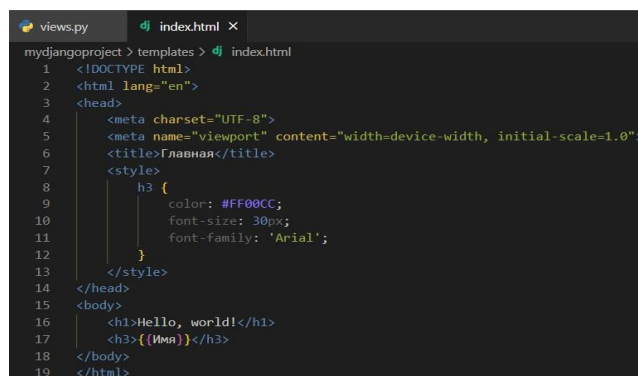
Рисунок 17 - Код, для того чтобы отобразить данные на странице

Обновим страницу. И что получим (рисунок 18)?



Рисунок 18 - Результат отображения на странице

С переданным аргументом можно делать все что угодно, например, изменять стили. Изменим код для тега head, добавив стили. И изменим немного шрифт, добавим данный текст в заголовок третьего уровня и, к примеру, добавим подчеркивание для текста (рисунок 19).



```

views.py  index.html X
mydjango project > templates > index.html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Главная</title>
7      <style>
8          h3 {
9              color: #FF00CC;
10             font-size: 30px;
11             font-family: 'Arial';
12         }
13     </style>
14 </head>
15 <body>
16     <h1>Hello, world!</h1>
17     <h3>{{Имя}}</h3>
18 </body>
19 </html>

```

Рисунок 19 - Добавление стилей

Проверим, как все отображается на странице и убедимся, что все работает (рисунок 20).

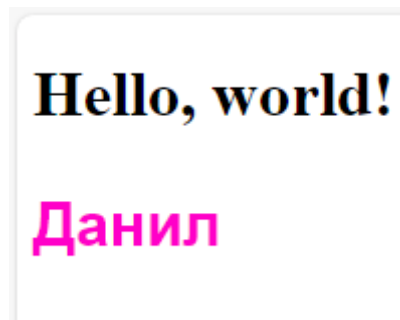


Рисунок 20 - Корректная работа стилей

Используя знания питона, несложно догадаться, что значения в словарь, можно передавать ещё и через переменную. Например (рисунок 21):

```
name = 'Данил'
return render(request, 'index.html', {'Имя': name})
```

Рисунок 21 - Передача значений в контексте в виде переменной

Или передать сразу словарь в виде переменной (рисунок 22):

```
about_me = {'Имя': 'Данил'}
return render(request, 'index.html', about_me)
```

Рисунок 22 - Передача словаря в виде контекста

Принцип думаю понятен. Можно передавать и списки, и вложенные словари. Создадим новый ключ `workers` и в качестве значения передадим список всех работников (вспоминаем предыдущую практическую работу). А также, в файле `index.html` поменяем название ключа (рисунок 23).

```
mydjango> project > app1 > views.py > ...
1  from django.shortcuts import render
2  from app1.models import Worker
3
4  def index_page(request):
5
6      all_workers = Worker.objects.all()
7
8      return render(request, 'index.html', {'workers': all_workers})
9
10
11  def about_page(request):
12      return render(request, 'about.html')
13
```

Рисунок 23 - Изменение названия ключа для списка всех сотрудников на `workers`

Но, что мы видим (рисунок 24)?

Hello, world!

```
<QuerySet [<Worker: Газизуллин Данил>, <Worker: Иванов Иван>, <Worker: Дементьев
Алексей>, <Worker: Петров Арсений>, <Worker: Захаров Михаил>]>
```

Рисунок 24 - Вывод данных на странице

В таком виде, нам нельзя выводить информацию пользователю. Для того, чтобы нормализовать вывод необходимо использовать язык шаблонов Django, или Django Template Language (Далее – DTL)

Django Template Language — текстовый язык шаблонов, который обеспечивает мост между сценариями (HTML, CSS, JS и т. д.) и языками программирования (Python)

По сути, шаблон Django – обычный html-файл, но с большими возможностями.

Что из этого следует понять? Стоит ли писать на языке шаблонов Django, как на отдельном языке программирования, нет, ни в коем случае! Можно реализовать циклы, условия и прочее с помощью данного языка. Засовывать в текст HTML какие-то немудрёные расчеты и программную логику – не нужно! По-хорошему все делается в бекенде и во фронтенд передавать уже готовые данные, но если прямо присутствует необходимость сделать логику во фронтенд-части, то используют JavaScript.

Реализуем цикл for для того, чтобы вывести каждого работника по отдельности (рисунок 25). Результат представлен на рисунке 26.

```
<body>
  <h1>Hello, world!</h1>

  {% for i in workers %}
    {{ i }}
  {% endfor %}

</body>
```

Рисунок 25 - Цикл for на языке DTL

Hello, world!

Газизуллин Данил Иванов Иван Дементьев Алексей Петров Арсений Захаров Михаил

Рисунок 26 - Отображение сотрудников

Отлично, но что делать, если все вывелось в одну строку и у нас нет оформления стилей для заголовка третьего уровня. Все просто, обернем нашу итерируемый объект в тег заголовка третьего уровня (рисунок 27).

```
<body>
  <h1>Hello, world!</h1>

  {% for i in workers %}
    <h3>{{ i }}</h3>
  {% endfor %}
</body>
```

Рисунок 27 - Корректировка цикла для отображения

И получим красивый результат (рисунок 28):

Hello, world!

Газизуллин Данил

Иванов Иван

Дементьев Алексей

Петров Арсений

Захаров Михаил

Рисунок 28 - Красивый вывод сотрудников

В DTL есть некая «внутренняя переменная», которая называется `forloop.counter`. Она считает количество итераций цикла (похожая функция в Python – `enumerate`). Добавим вывод № сотрудника перед каждой итерацией в заголовок второго уровня (рисунок 29).

```

<body>
  <h1>Список сотрудников</h1>

  {% for i in workers %}
    <h2>Сотрудник №{{ forloop.counter }}</h2>
    <h3>{{ i }}</h3>
  {% endfor %}
</body>

```

Рисунок 29 - Добавление счетчика итераций в цикл for

Приведем все в красивый вид и получим (рисунок 30):

```

<style>
  h2, h3 {
    font-family: 'Arial';
    font-size: 14px;
  }
  h2 {
    color: #4682B4;
  }
  h3 {
    color: #CD5C5C;
  }
</style>

```

Список сотрудников

Сотрудник №1
Газизуллин Данил

Сотрудник №2
Иванов Иван

Сотрудник №3
Дементьев Алексей

Сотрудник №4
Петров Арсений

Сотрудник №5
Захаров Михаил

Рисунок 30 - Корректировка стилей для отображения и результат

Если просмотреть исходный код страницы, все выглядит как обычные HTML-теги (рисунок 31).

```

<!DOCTYPE html>
<html lang="en">
  <head>
  </head>
  <body>
    <h1>Список сотрудников</h1>
    <h2>Сотрудник №1</h2>
    <h3>Газизуллин Данил</h3>
    <h2>Сотрудник №2</h2>
    <h3>Иванов Иван</h3>
    <h2>Сотрудник №3</h2>
    <h3>Дементьев Алексей</h3>
    <h2>Сотрудник №4</h2>
    <h3>Петров Арсений</h3>
    <h2>Сотрудник №5</h2>
    <h3>Захаров Михаил</h3>
    <div style="all: unset;">
  </div>
  </body>
</html>

```

Рисунок 31 - Исходный код страницы

А что будет, если мы в models.py удалим строчку кода, которая отвечает за определение строкового представления? (`_str_`). В данном случае, закомментируем её (рисунок 32).

```

mydjango> app1 > models.py > ...
1  from django.db import models
2
3  class Worker(models.Model):
4      name = models.CharField(max_length=20, blank=False)
5      surname = models.CharField(max_length=35, blank=False)
6      salary = models.IntegerField(default=0)
7
8      # def __str__(self) -> str:
9      #     return f"{self.surname} {self.name}"
10
11 class Equipment(models.Model):
12     name = models.CharField(max_length=50)
13     purchase_date = models.DateField()
14     price = models.IntegerField(default=0)

```

Рисунок 32 - Комментирование строкового представления для модели Worker

Если обновим страницу, увидим следующий беспорядок (рисунок 33):

Список сотрудников

Сотрудник №1
Worker object (1)
Сотрудник №2
Worker object (2)
Сотрудник №3
Worker object (3)
Сотрудник №4
Worker object (4)
Сотрудник №5
Worker object (5)

Рисунок 33 - Цикл с плохим отображением сотрудников

Поэтому, важно научиться обращаться конкретно напрямую к значению. Делается все точно также, как мы учились делать это через обращение к значению класса (рисунок 34).

```

<h1>Список сотрудников</h1>

{% for i in workers %}
    <h2>Сотрудник №{{ forloop.counter }}</h2>
    <h3>{{ i.surname }} {{ i.name }} {{ i.salary }}</h3>
{% endfor %}

```

Рисунок 34 - Редактирование цикла for

Также, забегаая вперед, отмечу, что точка, используется ещё для обращения по индексу. Если в питоне мы используем для этого квадратные скобки, то в DTL

необходимо писать через точку (рисунок 35).

```
users = ['Данил', 'Иван', 'Михаил']
print(users[0]) # Выведет Данил
```

Данил

Рисунок 35 - Обращение по индексу в списках в Python

Эквивалент на DTL (рисунок 36):

```
def index_page(request):
    all_workers = Worker.objects.all()
    users = ['Данил', 'Иван', 'Михаил']
    return render(request, 'index.html', {'users': users})
```

Рисунок 36 - Передача списка пользователей в контекст

В файле HTML пропишем название переданной переменной пользователей (users), а также обратимся по индексу к именам (рисунок 37).

```
<h1>Эквивалент</h1>
{{ users }} <br>
{{ users.0 }} <br>
{{ users.1 }} <br>
{{ users.2 }}
```

Эквивалент
 ['Данил', 'Иван', 'Михаил']
 Данил
 Иван
 Михаил

Рисунок 37 - Отображение списка пользователей, а также обращение к ним по индексу

Условно, давайте выделим зарплату сотрудников зеленым цветом, если она больше 50 000 рублей, если меньше, то красный.

Условие **if** в языке шаблонов Django позволяет условно отображать части шаблона на основе значений переменных, выражений или сравнений (рисунок 38). Результат представлен на рисунке 39.

```
{% for i in workers %}
  <h2>Сотрудник №{{ forloop.counter }}</h2>
  <h3>
    Фамилия: {{ i.surname }} <br>
    Имя: {{ i.name }} <br>
    Зарплата: {% if i.salary >= 50000 %}
      <b style='color: green'>{{ i.salary }}</b><br>
    {% endif %}
  </h3>
{% endfor %}
```

Рисунок 38 - Условие, при котором те сотрудники, у которых salary > 50000 – будут отображаться зелеными



Рисунок 39 - Отображение списка сотрудников с зарплатой выше 50000

На данном этапе мы видим, что если мы не добавим условие **else**, то зарплата тех сотрудников, которая меньше 50 000 рублей, не выведется (рисунок 40). Добавим блок **else**:

```
<h1>Список сотрудников</h1>

{% for i in workers %}
    <h2>Сотрудник №{{ forloop.counter }}</h2>
    <h3>
        Фамилия: {{ i.surname }} <br>
        Имя: {{ i.name }} <br>
        Зарплата: {% if i.salary >= 50000 %}
            <b style='color: green'>{{ i.salary }}</b><br>
        {% else %}
            <b style='color: red'>{{ i.salary }}</b><br>
        {% endif %}
    </h3>
{% endfor %}
```

Рисунок 40 - Добавление блока **else**

В конечном итоге, мы получим такой результат (рисунок 41):



Рисунок 41 - Корректное отображение списка сотрудников

Пример решения задачи на языке программирования Python

Задача. В ранее созданном проекте `blog_site` и приложении `posts` создайте папку `templates/posts`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`. Затем создайте шаблон `post_list.html`, который наследуется от `base.html` и выводит список публикаций блога из базы данных. Реализуйте представление `post_list` в файле `views.py`, передающее данные модели `Post` в шаблон. Настройте маршрут в файле `urls.py` для отображения списка публикаций и убедитесь, что страница корректно отображается в браузере.

Решение задачи:

1 Представление (`views.py`)

```
from django.shortcuts import render
from .models import Post
def post_list(request):
    posts = Post.objects.all()
    return render(request, 'posts/post_list.html', {'posts': posts})
```

2 Маршруты приложения (`posts/urls.py`)

```
from django.urls import path
from . import views
urlpatterns = [
    path("", views.post_list, name='post_list'),
]
```

3 Подключение маршрутов приложения (`blog_site/urls.py`)

```
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('posts.urls')),
]
```

4 Базовый шаблон (templates/posts/base.html)

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Блог</title>
</head>
<body>
    <h1>Блог публикаций</h1>
    <nav>
        <a href="/">Главная</a>
    </nav>
    <hr>
    {% block content %}
    {% endblock %}
</body>
</html>
```

5 Шаблон списка публикаций (templates/posts/post_list.html)

```
{% extends 'posts/base.html' %}
{% block content %}
<h2>Список публикаций</h2>
<ul>
    {% for post in posts %}
        <li>
            <strong>{{ post.title }}</strong><br>
            Автор: {{ post.author }}<br>
            Дата: {{ post.created_at }}
        </li>
        <br>
    {% empty %}
        <li>Публикации отсутствуют.</li>
    {% endfor %}
</ul>
```

```
{% endblock %}
```

6 Проверка работы

Запустите сервер:

```
python manage.py runserver
```

Откройте в браузере:

```
http://127.0.0.1:8000/
```

Работа со статическими файлами

Центральным моментом любого веб-приложения является обработка запроса, который отправляет пользователь. В Django за обработку запроса отвечают представления или `views`. По сути представления представляют функции обработки, которые принимают данные запроса в виде объекта `HttpRequest` из пакета `django.http` и генерируют некоторый результат, который затем отправляется пользователю.

Обычно в веб-приложениях используются различные статические файлы, такие как изображения, файлы `css` (для стилей), `javascript` и т.д. Давайте рассмотрим, как эти файлы могут быть использованы.

При создании проекта Django уже предоставляет базовые настройки для работы со статическими файлами. Например, в файле `settings.py` определена переменная `STATIC_URL`, которая содержит адрес каталога со статическими ресурсами (рисунок 1).

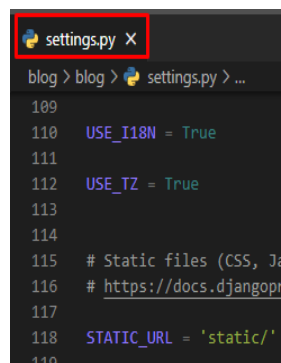


Рисунок 1 - Значение по умолчанию

А среди установленных приложений в переменной `INSTALLED_APPS` указано приложение `django.contrib.staticfiles` (рисунок 2).

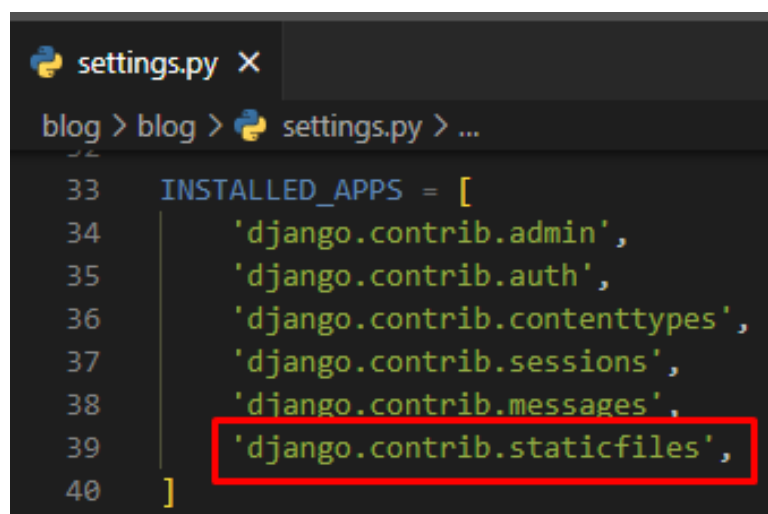


Рисунок 2 - `django.contrib.staticfiles` в `INSTALLED_APPS`

Переменная `STATIC_URL` установлена на `"static/"`, что означает, что мы можем просто создать каталог с названием `"static"` внутри папки нашего

приложения и добавить туда нужные статические ресурсы. Однако, при необходимости, мы всегда можем изменить расположение каталога статических файлов через эту настройку.

Итак, мы добавляем новый каталог "static" внутри нашего приложения. Чтобы не смешивать все статические файлы в одной папке, мы решаем организовать разные каталоги для различных видов файлов. Например, мы создаем каталог "images" внутри "static" для изображений, и отдельный каталог "css" для файлов стилей. Точно так же мы можем создать папки и для других видов файлов.

Давайте создадим следующую иерархию файлов и поместим фотографию ночного неба в папку images (рисунок 3):

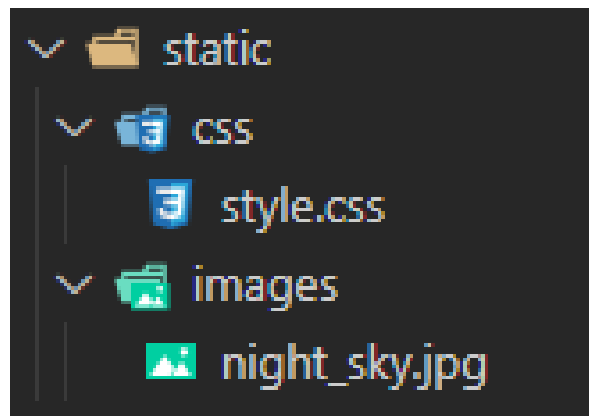


Рисунок 3 - Иерархия для статических файлов

В файле settings.py укажем каталог STATICFILES_DIRS.

STATICFILES_DIRS в **Django** — список дополнительных (нестандартных) путей к статическим файлам, используемых для сбора и для режима отладки (рисунок 4).

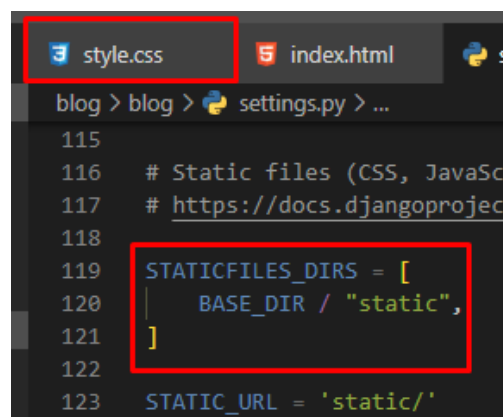


Рисунок 4 - Константа STATICFILES_DIRS

Создадим также папку templates и в ней файл index.html (рисунок 5).

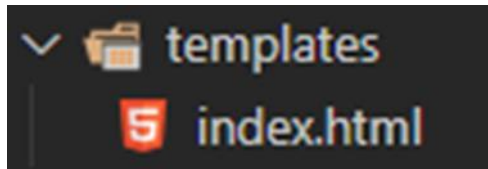


Рисунок 5 - Папка templates

Пропишем стили для сайта в файле стилей (рисунок 6).

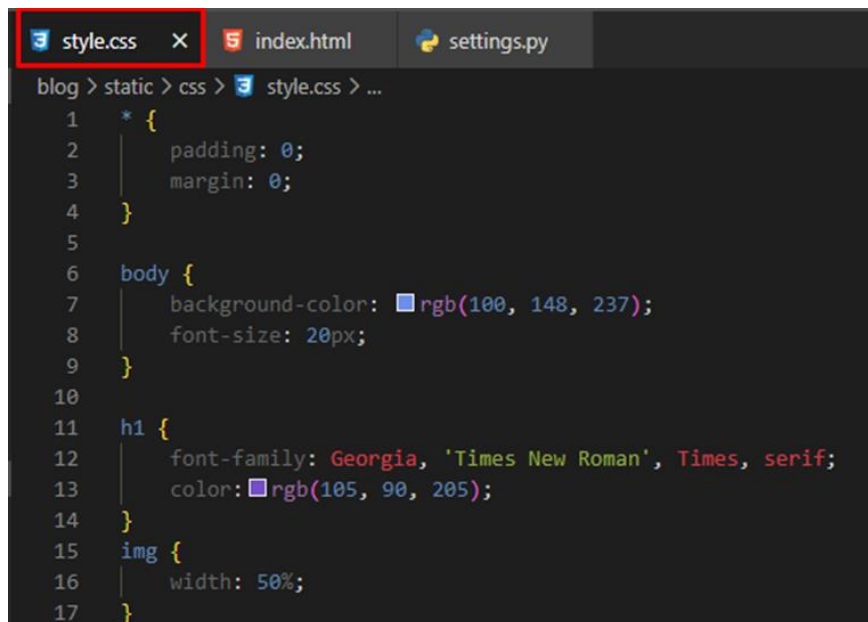


Рисунок 6 - Стили для страницы index.html

Подключим файл стилей к index.html, а также разместим в теге первого уровня текст «Ночное небо» и в теге изображений добавим путь к нашему изображению, используя все тот же язык DTL (Django Template Language). В качестве альтернативного текста (если картинки по каким-либо причинам не отобразится на страничке), установим все то же значение «Ночное небо» (рисунок 7).

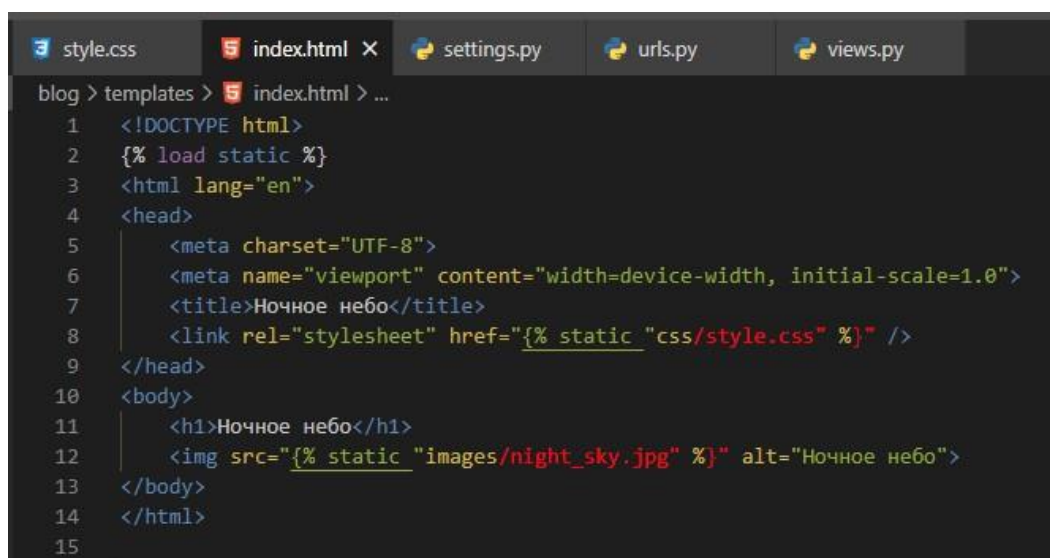


Рисунок 7 - Код для index.html

Результат нашей работы (рисунок 8):

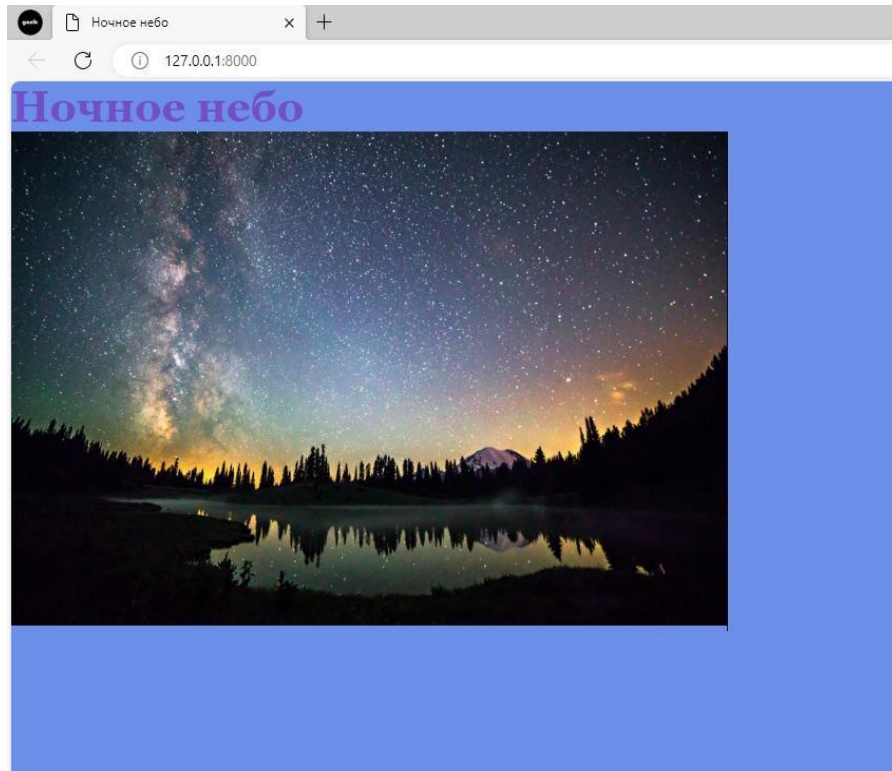


Рисунок 8 - Ночное небо с картинкой

Наследование шаблонов

Управляющими элементами шаблонов Django являются переменные, фильтры и теги.

При рендеринге шаблона переменные заменяются на свое значение, вычисленное в контексте вызова.

Синтаксис — двойные фигурные скобки, например: `{{ title }}`.

Фильтры служат для простых преобразований переменных.

Синтаксис — `переменная|имя_фильтра:"параметры"`. Фильтр может встречаться как в переменных, так и в качестве параметра тега.

Например: `{{ title|lowercase }}`.

При рендеринге шаблона теги, грубо говоря, заменяются на результаты работы ассоциированной с этим тегом функции на питоне.

Синтаксис: `{% тег параметры %}`

например: `{% url blog_article slug=article.slug %}`

Программист может написать свои фильтры и теги, но об этом позже. Кроме того, есть три специальных тега: `include`, `block` и `extend`.

Тег `include` подставляет запрошенный шаблон (отрендеренный в текущем контексте).

Все вышеперечисленное тривиально и в той или иной форме есть в любом

движке шаблонов. Теперь перейдем к особенностям Django: на тегах `block` и `extend` строится наследование шаблонов. Остановимся на них подробнее.

Основная фишка шаблонов Django — наследование. Шаблон может расширять (уточнять) поведение родительского шаблона.

Любой участок шаблона может быть обернут в блочный тег (естественно, что тег не может начинаться перед, а заканчиваться внутри цикла). Блоку дается имя. Например, блок контент (рисунок 9):

```
1  {% block content %}  
2      <!-- Содержимое страницы -->  
3  {% endblock content %}
```

Рисунок 9 - Пример использования блоков

При помощи тега `extend` мы указываем, какой шаблон мы будем уточнять. Расширяя шаблон, мы можем переопределить любые блоки, которые есть в родительском шаблоне. Все, что находится вне этих блоков, будет пропущено.

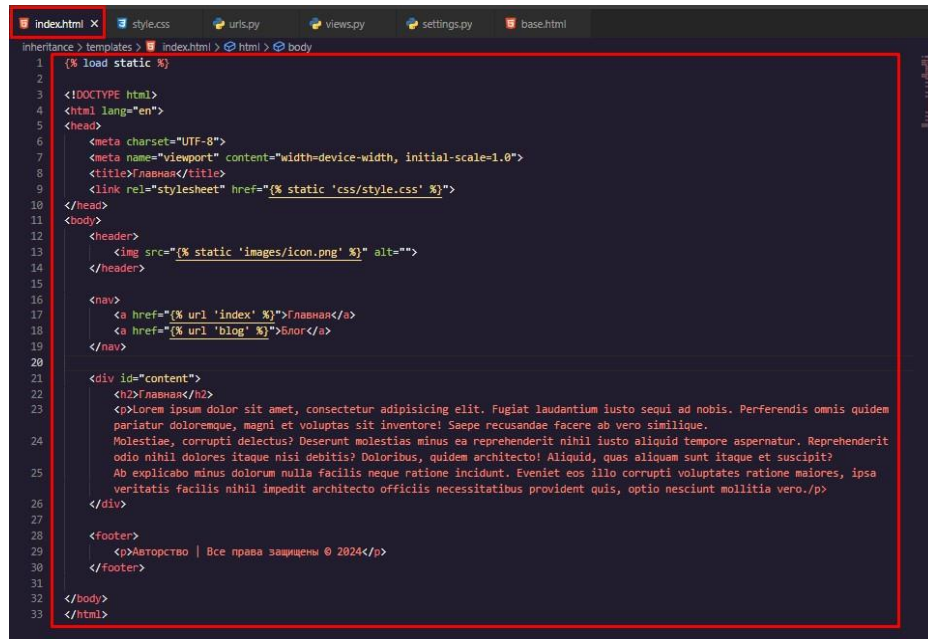
Получается мощный механизм, практически исключающий необходимость повторения частей шаблонов. Давайте разберем реальный пример.

Предположим, мы разрабатываем веб-сайт, включающий в себя как обычные страницы, так и блог. Верстальщик предоставил нам макет одной страницы сайта, содержащий:

- шапку (логотип, меню);
- тело страницы;
- и подвал с информацией о правах распространения.

В качестве основы для дальнейшей работы мы создадим шаблоны, используя исключительно HTML и CSS, это позволит нам сфокусироваться на структуре и стилизации, отложив использование других технологий на более поздний этап.

Так выглядит `index.html` (рисунок 10):



```

1  {% load static %}
2
3  <!DOCTYPE html>
4  <html lang="en">
5  <head>
6      <meta charset="UTF-8">
7      <meta name="viewport" content="width=device-width, initial-scale=1.0">
8      <title>Главная</title>
9      <link rel="stylesheet" href="{% static 'css/style.css' %}">
10 </head>
11 <body>
12     <header>
13         
14     </header>
15
16     <nav>
17         <a href="{% url 'index' %}">Главная</a>
18         <a href="{% url 'blog' %}">Блог</a>
19     </nav>
20
21     <div id="content">
22         <h2>Главная</h2>
23         <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Fugiat laudantium iusto sequi ad nobis. Perferendis omnis quidem
24         pariatur doloremque, magni et voluptas sit inventore! Saepe recusandae facere ab vero similique.
25         Molestiae, corrupti delectus? Deserunt molestias minus ea reprehenderit nihil iusto aliquid tempore aspernatur. Reprehenderit
26         odio nihil dolores itaque nisi debitis? Doloribus, quidem architecto! Aliquid, quas aliquam sunt itaque et suscipit?
27         Ab explicabo minus dolorum nulla facilis neque ratione incidunt. Eveniet eos illo corrupti voluptates ratione maiores, ipsa
28         veritatis facilis nihil impedit architecto officiis necessitatibus provident quis, optio nesciunt mollitia vero.</p>
29     </div>
30
31     <footer>
32         <p>Авторство | Все права защищены © 2024</p>
33     </footer>
34 </body>
35 </html>

```

Рисунок 10 - Файл index.html

Так выглядит style.css (рисунок 11):



```

1  body {
2      font-family: Arial, sans-serif;
3      margin: 0;
4      padding: 0;
5  }
6
7  header {
8      background-color: #333;
9      color: #fff;
10     padding: 10px 0;
11     text-align: center;
12 }
13
14 header img {
15     width: 100px;
16 }
17
18 header h1 {
19     margin: 0;
20 }
21
22 #content h2 {
23     text-align: center;
24 }
25
26 #content p {
27     text-align: center;
28     margin: 0 150px;
29     line-height: 1.2;
30 }
31
32 nav {
33     background-color: #444;
34     text-align: center;
35     padding: 10px;
36 }
37
38 nav a {
39     color: #fff;
40     text-decoration: none;
41     padding: 10px;
42 }
43
44 nav a:hover {
45     background-color: #666;
46 }
47
48 footer {
49     background-color: #333;
50     color: #fff;
51     text-align: center;
52     padding: 10px 0;
53     position: fixed;
54     bottom: 0;
55     width: 100%;
56 }

```

Рисунок 11 - Файл стилей (style.css)

Теперь, создадим страницу блога (рисунок 12):

```

1  """
2  URL configuration for inheritance project.
3
4  The `urlpatterns` list routes URLs to views. For more information please see:
5  https://docs.djangoproject.com/en/5.0/topics/http/urls/
6  Examples:
7  Function views
8      1. Add an import: from my_app import views
9      2. Add a URL to urlpatterns: path('', views.home, name='home')
10 Class-based views
11     1. Add an import: from other_app.views import Home
12     2. Add a URL to urlpatterns: path('', Home.as_view(), name='home')
13 Including another URLconf
14     1. Import the include() function: from django.urls import include, path
15     2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))
16 """
17 from django.contrib import admin
18 from django.urls import path
19
20 from index.views import index, blog
21
22 urlpatterns = [
23     path('admin/', admin.site.urls),
24     path('', index, name='index'),
25     path('blog', blog, name='blog'),
26 ]
27

```

Рисунок 12 - Файл blog.html

Настроим url-шаблоны, для правильного перехода между страницами (рисунок 13).

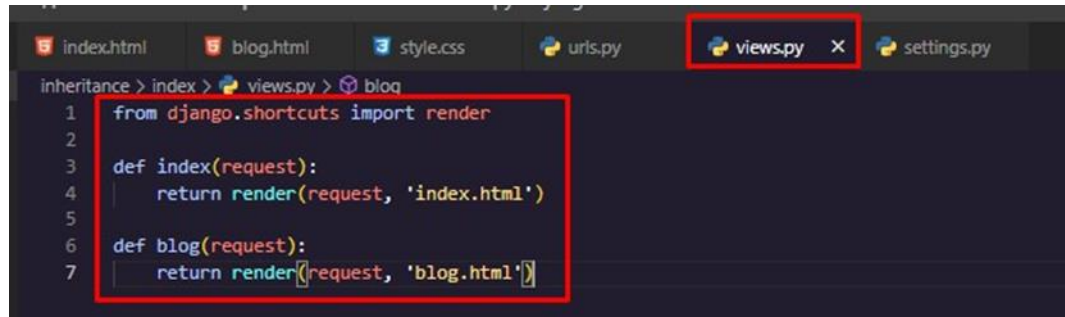
```

1  {% load static %}
2
3  <!DOCTYPE html>
4  <html lang="en">
5  <head>
6      <meta charset="UTF-8">
7      <meta name="viewport" content="width=device-width, initial-scale=1.0">
8      <title>Блог</title>
9      <link rel="stylesheet" href="{% static 'css/style.css' %}">
10 </head>
11 <body>
12     <header>
13         
14     </header>
15
16     <nav>
17         <a href="{% url 'index' %}">Главная</a>
18         <a href="{% url 'blog' %}">Блог</a>
19     </nav>
20
21     <div id="content">
22         <h2>Какая-то информация</h2>
23         <p>Содержание</p>
24     </div>
25
26     <footer>
27         <p>Авторство | Все права защищены © 2024</p>
28     </footer>
29 </body>
30 </html>
31
32

```

Рисунок 13 - Файл urls.py

Настроим отображение страниц в файле представлений (views.py) (рисунок 14).



```

1 from django.shortcuts import render
2
3 def index(request):
4     return render(request, 'index.html')
5
6 def blog(request):
7     return render(request, 'blog.html')

```

Рисунок 14 - Файл views.py

И в конечном итоге мы получаем (рисунки 15-16):

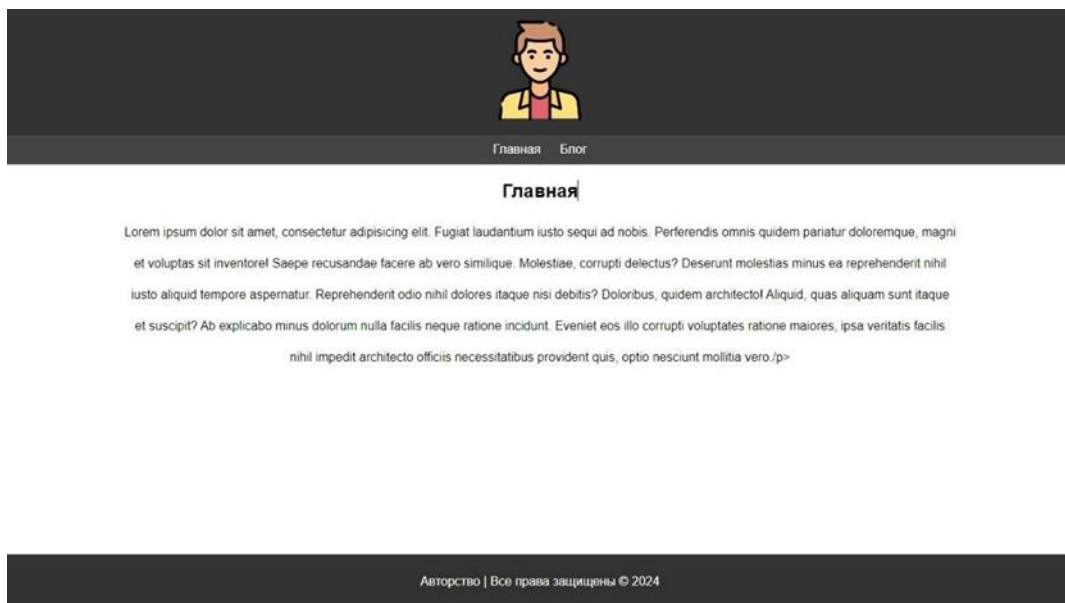


Рисунок 15 - Итоговый вид главной страницы

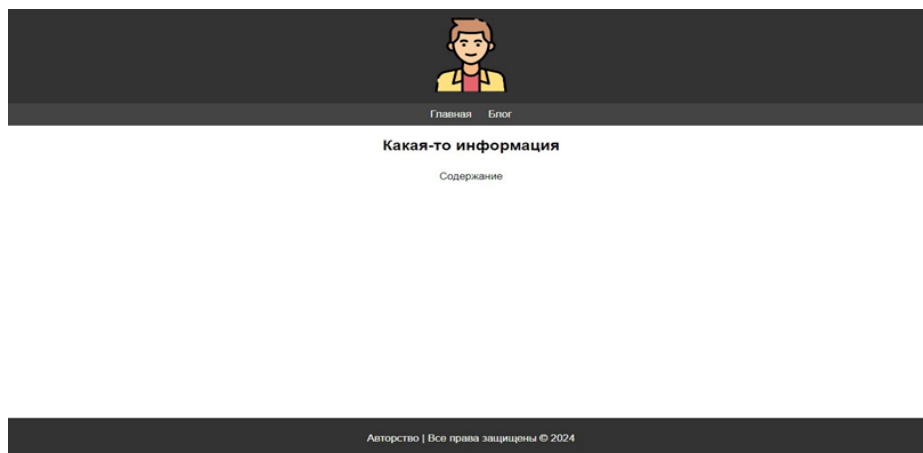
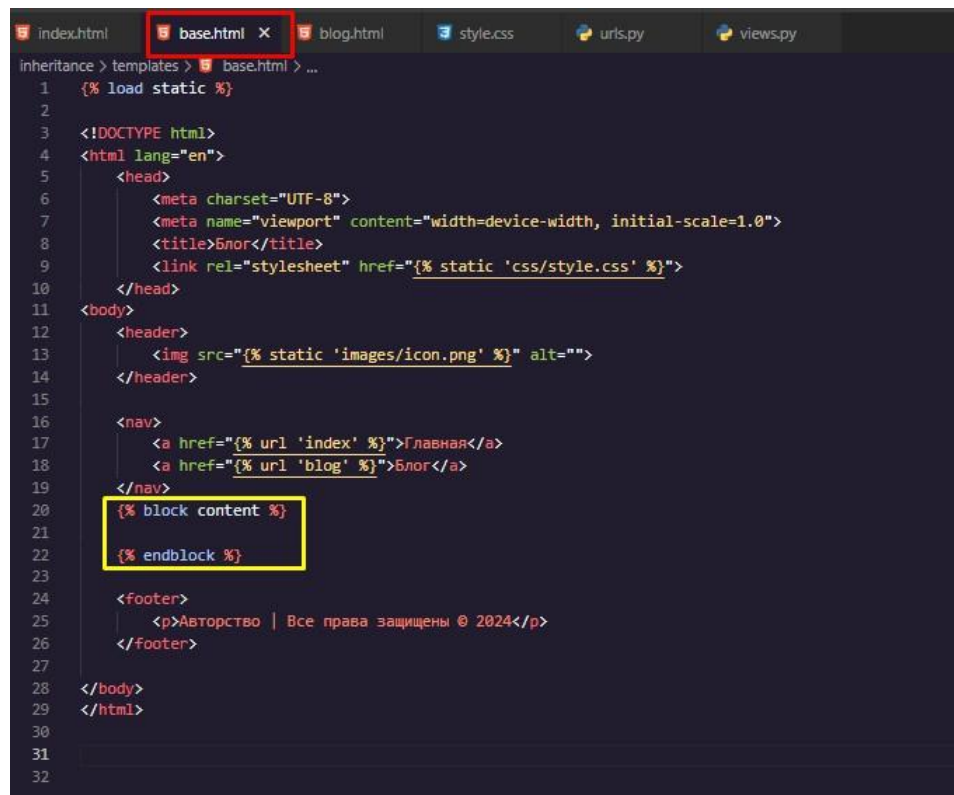


Рисунок 16 - Итоговый вид страницы blog.html

Если Вы заметили, то код index.html практически идентичен коду blog.html. Создадим базовый шаблон под названием base.html и поместим, согласно

теоретической части практической работы, код используя блоки. Добавим блок content, дабы в будущем мы могли расширять данный html-файл в других html-файлах (рисунок 17).



```

1  {% load static %}
2
3  <!DOCTYPE html>
4  <html lang="en">
5      <head>
6          <meta charset="UTF-8">
7          <meta name="viewport" content="width=device-width, initial-scale=1.0">
8          <title>Блог</title>
9          <link rel="stylesheet" href="{% static 'css/style.css' %}">
10     </head>
11     <body>
12         <header>
13             
14         </header>
15
16         <nav>
17             <a href="{% url 'index' %}">Главная</a>
18             <a href="{% url 'blog' %}">Блог</a>
19         </nav>
20         {% block content %}
21         {% endblock %}
22
23         <footer>
24             <p>Авторство | Все права защищены © 2024</p>
25         </footer>
26
27     </body>
28 </html>
29
30
31
32

```

Рисунок 17 - Базовый шаблон base.html

Изменим код index.html на следующий (рисунок 18):



```

1  {% extends "base.html" %}
2
3  {% block content %}
4      <div id="content">
5          <h2>Главная</h2>
6          <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fugiat laudantium iusto sequi ad nobis. Perferendis omnis quidem
7              pariatur doloremque, magni et voluptas sit inventore! Saepe recusandae facere ab vero similique.
8              Molestiae, corrupti delectus? Deserunt molestias minus ea reprehenderit nihil iusto aliquid tempore aspernatur. Reprehenderit
9              odio nihil dolores itaque nisi debitis? Doloribus, quidem architecto! Aliquid, quas aliquam sunt itaque et suscipit?
10             Ab explicabo minus dolorum nulla facilis neque ratione incidunt. Eveniet eos illo corrupti voluptates ratione maiores, ipsa
11             veritatis facilis nihil impedit architecto officiis necessitatibus provident quis, optio nesciunt mollitia vero.</p>
12         </div>
13     {% endblock %}
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32

```

Рисунок 18 - Измененный index.html

Изменим код about.html на следующий (рисунок 19):

```

1 {% extends "base.html" %}
2
3 {% block content %}
4 <div id="content">
5     <h2>Какая-то информация</h2>
6     <p>Содержание</p>
7 </div>
8 {% endblock %}

```

Рисунок 19 - Измененный blog.html

Перейдем на веб-сайт и посмотрим, что же изменилось (рисунки 20-21).



Рисунок 20 - Конечный результат страницы index.html

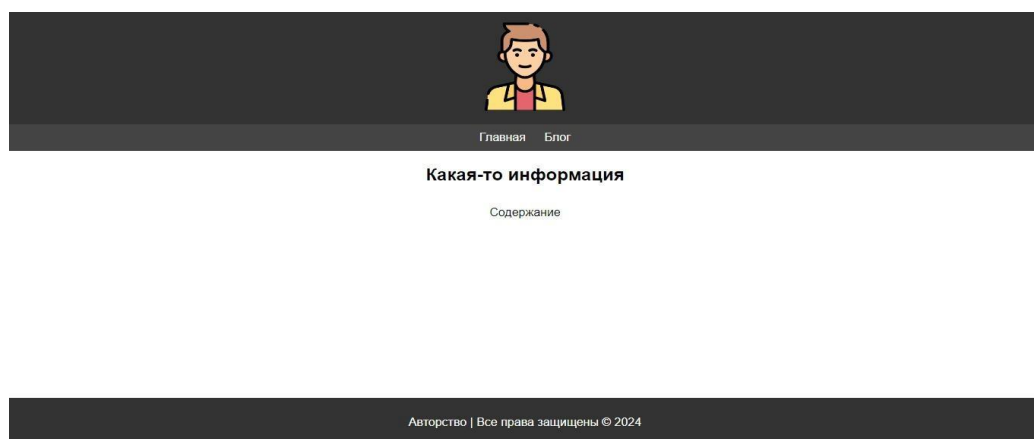


Рисунок 21 - Конечный результат страницы blog.html

Ничего не поменялось, потому что и не должно было меняться, ведь мы отображали на сайте ту же самую структуру, но более правильно, не дублируя код. При надобности, мы можем добавить ещё блоки и использовать их как мы хотим.

Передача данных в шаблоны

Одним из преимуществ шаблонов является то, что мы можем передать в них динамически из представлений различные данные. Для вывода данных в шаблоне могут использоваться различные способы. Для вывода самых простых данных применяется двойная пара фигурных скобок:

```
{{ название_объекта }}
```

Например, пусть в проекте у нас есть папка templates, в которой содержится шаблон index.html:

Определим в файле index.html следующий код:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Django на METANIT.COM</title>
</head>
<body>
  <h2>{{ header }}</h2>
  <p>{{ message }}</p>
</body>
</html>
```

Здесь используется две переменных: message и header. Они будут передаваться из представления.

Чтобы из функции-представления передать данные в шаблон применяется третий параметр функции render, который еще называется context и который представляет словарь. Например, изменим файл views.py следующим образом:

```
from django.shortcuts import render
def index(request):
    data = {"header": "Hello Django", "message": "Welcome to Python"}
    return render(request, "index.html", context=data)
```

В шаблоне используются две переменных, соответственно словарь, который передается в функцию render через параметр context, теперь содержит два значения с ключами header и message.

В результате при обращении к корню веб-приложения мы увидим следующий вывод в браузере (рисунок 22):

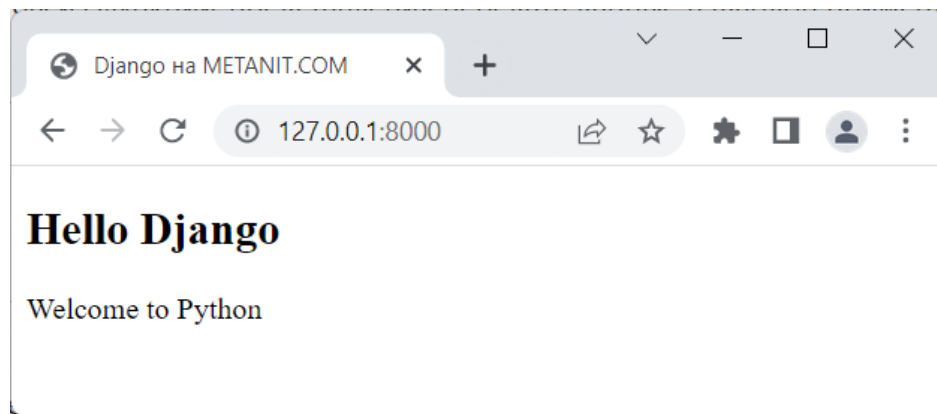


Рисунок 22 - Передача данных в шаблоны Templates в веб-приложении на Django

Рассмотрим передачу более сложных данных. Допустим, в представлении передаются следующие данные:

```
from django.shortcuts import render
def index(request):
    header = "Данные пользователя"          # обычная переменная
    langs = ["Python", "Java", "C#"]         # список
    user = {"name": "Tom", "age": 23}        # словарь
    address = ("Абрикосовая", 23, 45)       # кортеж
    data = {"header": header, "langs": langs, "user": user, "address": address}
    return render(request, "index.html", context=data)
```

В качестве третьего параметра в функцию `render` нам надо передать словарь, поэтому все данные оборачиваются в словарь и в таком виде передаются в шаблон.

В этом случае шаблон мог бы выглядеть, например, следующим образом:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Django на METANIT.COM</title>
</head>
<body>
  <h1>{{ header }}</h1>
  <p>Имя: {{ user.name }} Возраст: {{ user.age }}</p>
  <p>Адресс: ул. {{ address.0 }}, д. {{ address.1 }}, кв. {{ address.2 }}</p>
  <p>Языки: {{ langs.0 }}, {{ langs.1 }}</p>
</body>
</html>
```

Поскольку объекты `langs` и `address` представляют соответственно массив и кортеж, то мы можем обратиться к их элементам через индексы, как мы бы работали бы с ними в коде на Python, например, первый элемент кортежа `address`: `address.0`.

Подобным образом, поскольку объект `user` представляет словарь, то мы можем обратиться к его элементам по ключам `name` и `age`: `{{ user.name }}` `{{ user.age }}`.

В итоге мы получим следующий вывод в веб-браузере (рисунок 23):

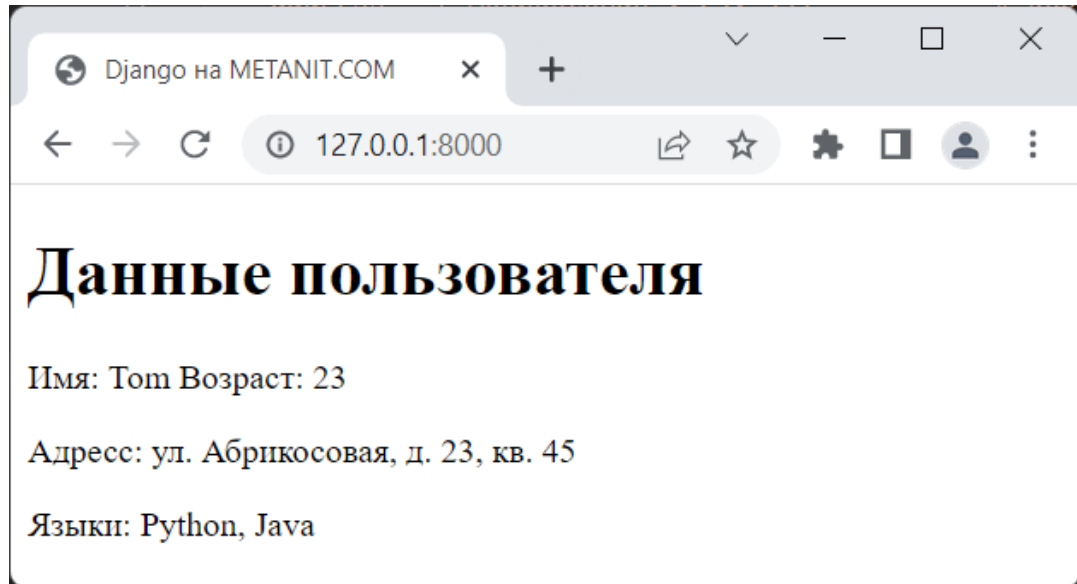


Рисунок 23 - Передача списков, кортежей и словарей в шаблон `template` в Django

Если для генерации шаблона применяется класс `TemplateResponse`, то в его конструктор также через третий параметр можно передать данные для шаблона:

```
from django.template.response import TemplateResponse
def index(request):
    header = "Данные пользователя"          # обычная переменная
    langs = ["Python", "Java", "C#"]        # список
    user = {"name": "Tom", "age": 23}        # словарь
    address = ("Абрикосовая", 23, 45)       # кортеж
    data = {"header": header, "langs": langs, "user": user, "address": address}
    return TemplateResponse(request, "index.html", data)
```

Подобным образом можно передавать в шаблоны объекты своих классов. Например, определение функции-представления (рисунок 24):

```
from django.shortcuts import render
def index(request):
    return render(request, "index.html", context = {"person": Person("Tom")})
class Person:
    def __init__(self, name):
        self.name = name # имя человека
```

Здесь определяется класс `Person`, в конструкторе которого передается некоторое значение для атрибута `name`. В функции `index` в шаблон передается объект с ключом `"person"`.

В шаблоне `index.html` мы можем обращаться к функциональности объекта, например, к его атрибуту `name`:


```

<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Django на METANIT.COM</title>
</head>
<body>
  <h1>Person {{ person.name }}</h1>
</body>
</html>

```

Передача объектов классов в шаблон template в Django.

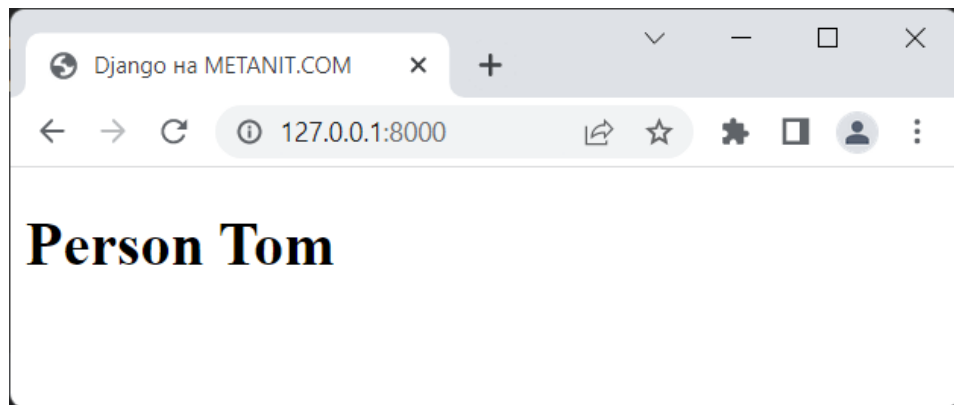


Рисунок 24 - Передача объектов классов в шаблон template в Django

Статические файлы

Веб-приложение, как правило, использует различные статические файлы - изображения, файлы стилей css, скриптов javascript и так далее. Рассмотрим, как мы можем использовать подобные файлы.

При создании проекта Django он уже имеет некоторую базовую настройку для работы со статическими файлами. В частности, в файле settings.py определена переменная STATIC_URL, которая хранит путь к каталогу со статическими файлами:

```
STATIC_URL = 'static/'
```

А среди установленных приложений в переменной INSTALLED_APPS указано приложение django.contrib.staticfiles (рисунок 25):

```

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'hello',
]

```

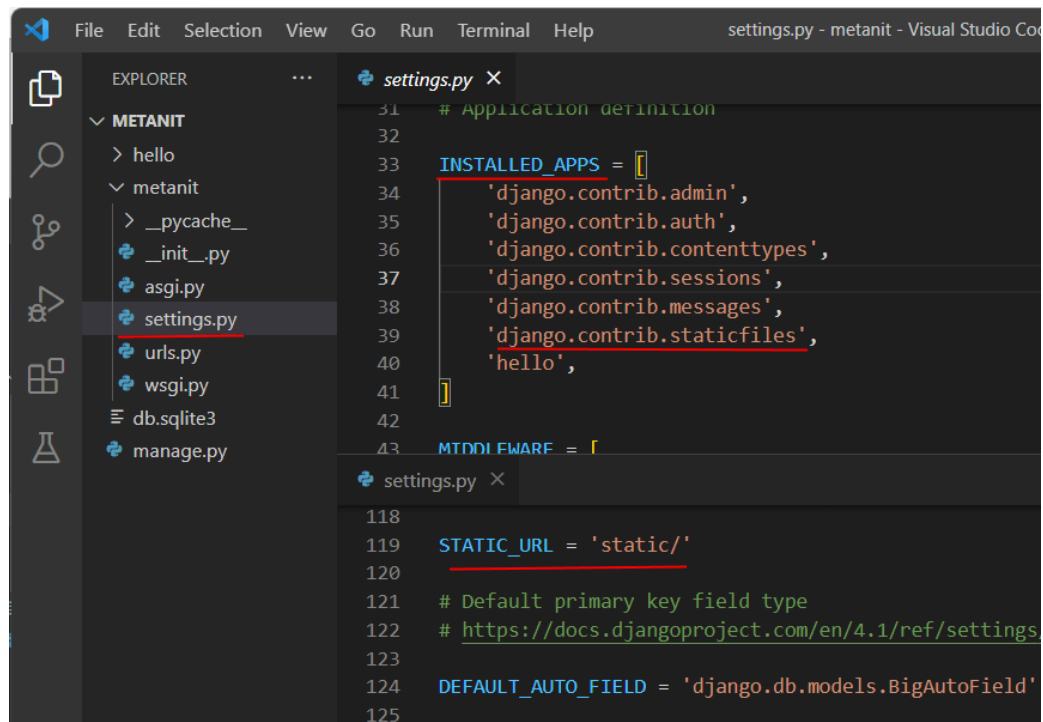


Рисунок 25 - Настройка статических файлов в файле settings.py в проекте на Django

Переменная `STATIC_URL` имеет значение `"static/"`, а это значит, что нам достаточно создать в папке приложения каталог с именем `"static"` и добавить в него необходимые нам статические файлы. Но, естественно, при необходимости через данную настройку мы можем изменить расположение каталога статических файлов.

Итак, добавим в папку приложения новый каталог `static`. Чтобы не сваливать все статические файлы в кучу, определим для каждого типа файлов отдельные папки. В частности, создадим в папке `static` для изображений каталог `images`, а для стилей - каталог `css`. Подобным образом можно создавать папки и для других типов файлов.

В папку `static/images` добавим какое-нибудь изображение - в моем случае это будет файл `forest.jpg`. А в папке `static/css` определим новый файл `styles.css`, который будет иметь какие-нибудь простейшие стили, например (рисунок 26):

```

body{ font-family: Verdana;}
h1{color:navy;}
img{width:350px;}

```

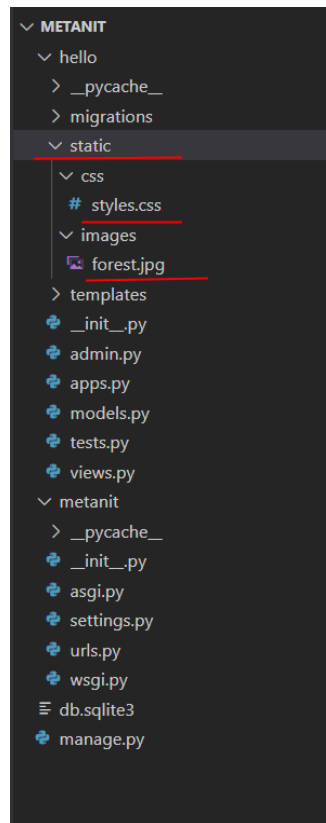


Рисунок 26 - Статические файлы в проекте на Django

Теперь используем эти файлы в шаблоне. Для этого в начале файла шаблона необходимо определить инструкцию

```
{% load static %}
```

При этом данный код должен идти после тега DOCTYPE.

Для определения пути к статическим файлам используются выражения типа

```
{% static "путь к файлу внутри папки static" %}
```

Так, пусть в приложении в папке templates определен шаблон index.html, который имеет следующий код (рисунок 27):

```
<!DOCTYPE html>
{% load static %}
<html>
<head>
  <meta charset="utf-8" />
  <link rel="stylesheet" href="{% static 'css/styles.css' %}" />
  <title>Django на METANIT.COM</title>
</head>
<body>
  <h1>Зимний лес</h1>
  
</body>
</html>
```

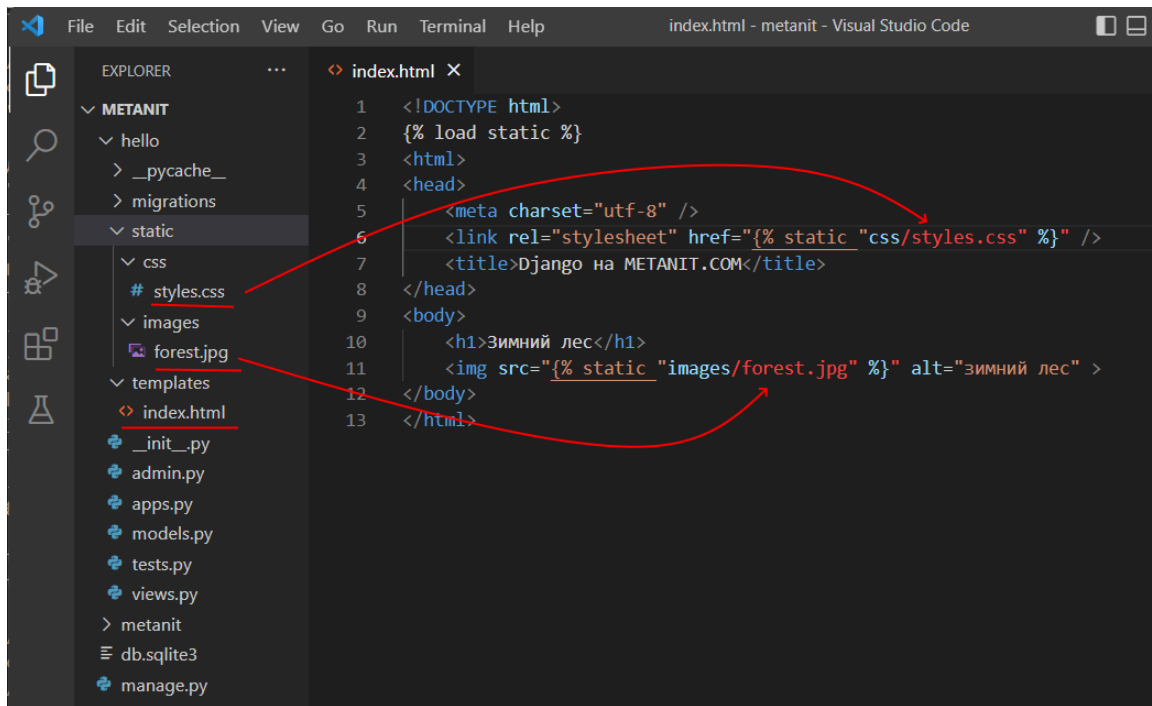


Рисунок 27 - Работа со статическими файлами в веб-приложении на Django

При запуске приложения шаблон `index.html` будет генерироваться в следующую веб-страницу, которая будет использовать изображение и применять стили (рисунок 28):

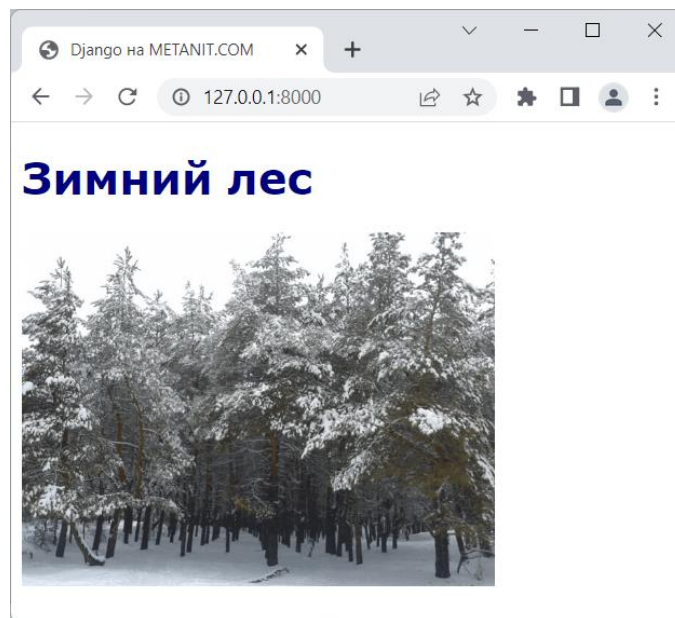


Рисунок 28 - Подключение статических файлов в приложении на Django и Python

Если нас не устраивает хранение файлов в каталоге по умолчанию - каталоге `static`, либо мы хотим указать несколько папок, то мы можем в файле `settings.py` задать все необходимые каталоги с помощью переменной `STATICFILES_DIRS`, которая принимает список путей:

```

STATICFILES_DIRS = [
    BASE_DIR / "static",
    "/var/www/static/",
    "/somefolder/"
]

```

Пример решения задачи на языке программирования Python

Задача. В ранее созданном проекте `blog_site` и приложении `posts` создайте папку `static/posts/css`. В этой папке создайте файл стилей `style.css`. Подключите созданный файл стилей в шаблоне `base.html` с помощью тегов `{% load static %}` и `{% static %}`. Убедитесь, что шаблон `post_list.html` наследуется от `base.html` с помощью тега `{% extends %}` и использует блок `{% block content %}`. Добавьте в файл CSS стили для оформления заголовка сайта, меню навигации и списка публикаций блога. Проверьте корректность отображения страницы в браузере.

Решение задачи:

1 Создание структуры каталогов

```

posts/
├── static/
│   ├── posts/
│   │   └── css/
│   │       └── style.css
├── templates/
│   └── posts/
│       ├── base.html
│       └── post_list.html

```

2 Файл стилей (`static/posts/css/style.css`)

```

body {
    font-family: Arial, sans-serif;
    margin: 20px;
}
h1 {
    color: navy;
}
nav {
    margin-bottom: 20px;
}
nav a {
    text-decoration: none;
    margin-right: 15px;
    color: darkgreen;
}
ul {
    list-style-type: none;
    padding: 0;
}

```

```

}
li {
    padding: 10px;
    border-bottom: 1px solid #cccccc;
}
3 Базовый шаблон (templates/posts/base.html)
{% load static %}
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Блог</title>
    <link rel="stylesheet" href="{% static 'posts/css/style.css' %}">
</head>
<body>
<h1>Блог публикаций</h1>
<nav>
    <a href="/">Главная</a>
</nav>
{% block content %}
{% endblock %}
</body>
</html>
4 Шаблон списка публикаций (templates/posts/post_list.html)
{% extends 'posts/base.html' %}
{% block content %}
<h2>Список публикаций</h2>
<ul>
    {% for post in posts %}
        <li>
            <strong>{{ post.title }}</strong><br>
            Автор: {{ post.author }}
        </li>
    {% empty %}
        <li>Публикации отсутствуют.</li>
    {% endfor %}
</ul>
{% endblock %}
5 Представление (views.py)
from django.shortcuts import render
from .models import Post
def post_list(request):
    posts = Post.objects.all()
    return render(request, 'posts/post_list.html', {'posts': posts})
6 Маршруты (posts/urls.py)

```

```

from django.urls import path
from .views import post_list
urlpatterns = [
    path("", post_list, name='post_list'),
]

```

7 Запуск проекта

```
python manage.py runserver
```

После открытия адреса <http://127.0.0.1:8000/> должна отображаться страница со списком публикаций блога, оформленная с использованием подключённого файла стилей style.css.

Описание задания на основе темы «Онлайн-магазин»

Главная страница

Основная страница вашего сайта. Должна содержать краткое описание сайта и ссылки на второстепенные страницы.

Три второстепенные страницы

Каждая второстепенная страница должна содержать информацию, соответствующую выбранной теме.

Исходя из темы, страницами могут быть: «Каталог товаров», «О нас», «Контакты».

Вывод данных (*data_view*)

На одной из второстепенных страниц должен быть размещен вывод данных. Вы должны реализовать вывод списка товаров, добавленных через административную панель Django. Вывод должен содержать информацию о каждом товаре, такую как имя, описание и цена.

Означает, что нужно создать модель товара с полями «Имя», «Описание» и «Цена», провести миграции и в админ-панели добавить товары. **Должно быть минимум 7 товаров.**

Статические файлы

Обязательным требованием является добавление стилей через отдельный файл стилей (style.css), т.е. нельзя прописывать стили внутри html-файла.

Также, реализовать картинки-ссылки (при нажатии на картинку, она должна открываться в НОВОМ ОКОШКЕ).

Подсказка: target "_blank"

Header, main и footer Каждая страница должна содержать:

- header (навигационное меню);
- main (основное содержание страницы);
- footer (авторство и тема сайта).

Примечание

Для реализации перехода между страницами, нужно использовать name в url_patterns.

В файле views.py укажем функцию, для отображения представления (рисунок 1).

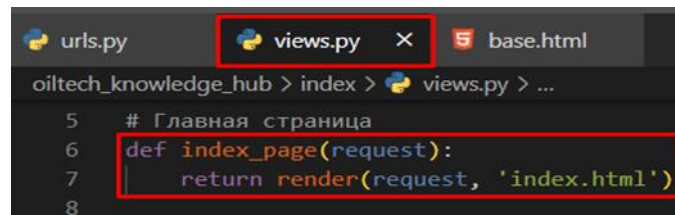


Рисунок 1 - View-функция для отображения страницы index.html

Импортируем функцию для отображения представления главной страницы в файл urls.py (рисунок 2):

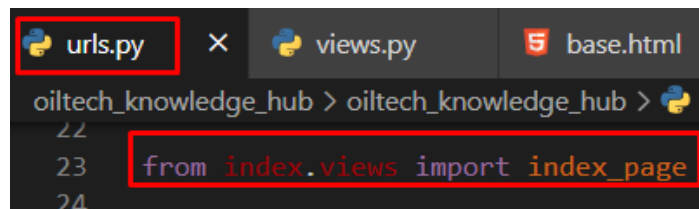


Рисунок 2 - Импорт view-функции в файле urls.py

В шаблонах URL-адресов укажем такой параметр, как name, по нему мы и будем ссылаться в html-файлах (рисунок 3).

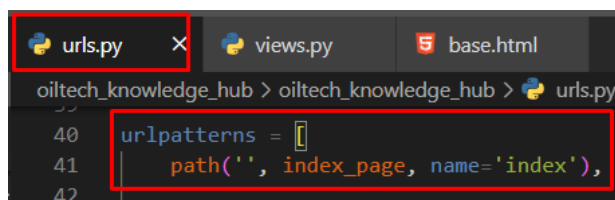


Рисунок 3 - Использование неймингов (name)

В файле index.html пропишем такую вот конструкцию, и все, теперь, ссылка будет ссылаться на файл index.html. Таким же образом, можно сделать переход по любому файлу (рисунок 4).

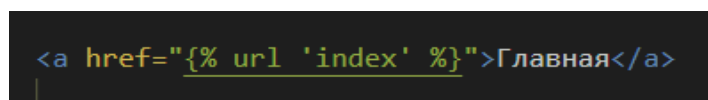


Рисунок 4 - Задание url-адреса в HTML шаблоне

Добавление стилей Bootstrap к сайту

CSS-фреймворк – набор файлов (CSS – Cascading style sheets, JS – JavaScript), подключаемый к HTML-странице и содержащий наборы стилей. Он нужен, чтобы ускорить и упростить процесс верстки. CSS-фреймворк содержит заранее написанный набор стилей, которые можно применять к элементам на сайте.

Таким образом, стилизовать элементы на сайте можно просто добавлением определенных классов. Наиболее известный CSS-фреймворк – Bootstrap.

Bootstrap включает в себя HTML и CSS-шаблоны оформления для текста, форм, кнопок, меток, блоков навигации и прочих компонентов веб-интерфейса.

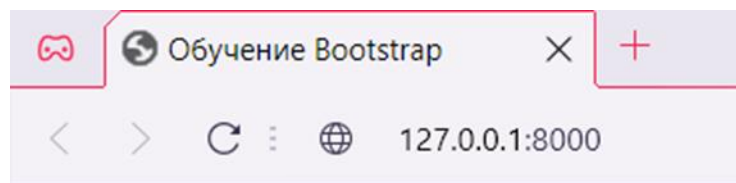
Есть несколько вариантов подключения к Django-сайту:

- подключить непосредственно через CDN (через ссылку);
- статическое подключение (т.е. скачивание папки Bootstrap с официального сайта);
- подключить Bootstrap с помощью python-модуля (django-bootstrap5). Его мы рассматривать не будем.

CDN (Content Delivery Network) – сеть доставки контента. Сервис сетевой передачи данных пользователям, независимо от их расположения

Для начала, самостоятельно разверните проект. Это значит, что Вам нужно:

- создать Django-проект и виртуальное окружение;
- установить Django;
- создать приложение и реализовать логику отображения страницы;
- создать папку templates и настроить её;
- запускаем сервер и смотрим, что все работает. В конечном итоге, Вы должны получить что-то подобное (рисунок 5).



Привет!

Рисунок 5 - Правильно развернутый проект

Подключим Bootstrap через CDN. Ссылка на сайт Bootstrap: <https://getbootstrap.com/docs/5.3/getting-started/introduction/> Заходим на сайт Bootstrap и копируем Quick start (рисунок 6).

```

bootstrap_learning > templates > main.html > html > head > meta
1 <!doctype html>
2 <html lang="en">
3
4 <head>
5   <meta charset="utf-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1">
7   <title>Bootstrap demo</title>
8   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet"
9     integrity="sha384-QMtkZyPPTpPRtY6eGknYGfeSXhJyWUU3f0f8OatM1R7y4tVPZSc4N3lHXoM4" crossorigin="anonymous">
10 </head>
11
12 <body>
13   <h1>Hello, world!</h1>
14   <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
15     integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5S9FVZLESaAA55NDz0xhy9GkciDs10IknN768169u"
16     crossorigin="anonymous"></script>
17 </body>
18
19 </html>

```

Рисунок 6 - Базовый шаблон html с подключением Bootstrap

Обновляем страницу и смотрим, что все работает (рисунок 7):

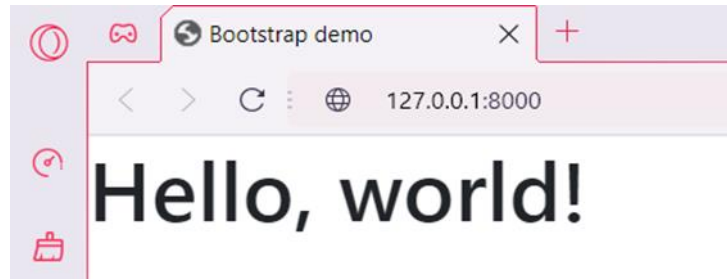


Рисунок 7 - Работающий Bootstrap через CDN подключение

Использование CDN не всегда хорошо подходит, потому что он, честно говоря, тормозит загрузку страницы.

Статическое подключение

Скачайте [архив](#) с сайта bootstrap

Из всего этого архива, нам пока что нужны два файла (рисунок 8):

- bootstrap.min.css;
- bootstrap.min.js.

bootstrap.css	20.02.2024 18:04	CSSfile	275 КБ
bootstrap.css.map	20.02.2024 18:04	Linker Address Map	664 КБ
bootstrap.min.css	20.02.2024 18:04	CSSfile	228 КБ
bootstrap.min.css.map	20.02.2024 18:04	Linker Address Map	577 КБ
bootstrap.rtl.css	20.02.2024 18:04	CSSfile	274 КБ
bootstrap.rtl.css.map	20.02.2024 18:04	Linker Address Map	664 КБ
bootstrap.rtl.min.css	20.02.2024 18:04	CSSfile	228 КБ
bootstrap.rtl.min.css.map	20.02.2024 18:04	Linker Address Map	576 КБ
bootstrap-grid.css	20.02.2024 18:04	CSSfile	69 КБ
bootstrap-grid.css.map	20.02.2024 18:04	Linker Address Map	199 КБ
bootstrap-grid.min.css	20.02.2024 18:04	CSSfile	51 КБ

Рисунок 8 - Файл bootstrap.min.css

В папке js находим файл bootstrap.min.js (рисунок 9):

bootstrap.bundle.js	20.02.2024 18:04	JSFile	203 KБ
bootstrap.bundle.js.map	20.02.2024 18:04	Linker Address Map	435 KБ
bootstrap.bundle.min.js	20.02.2024 18:04	JSFile	79 KБ
bootstrap.bundle.min.js.map	20.02.2024 18:04	Linker Address Map	325 KБ
bootstrap.esm.js	20.02.2024 18:04	JSFile	133 KБ
bootstrap.esm.js.map	20.02.2024 18:04	Linker Address Map	299 KБ
bootstrap.esm.min.js	20.02.2024 18:04	JSFile	73 KБ
bootstrap.esm.min.js.map	20.02.2024 18:04	Linker Address Map	218 KБ
bootstrap.js	20.02.2024 18:04	JSFile	142 KБ
bootstrap.js.map	20.02.2024 18:04	Linker Address Map	300 KБ
bootstrap.min.js	20.02.2024 18:04	JSFile	60 KБ
bootstrap.min.js.map	20.02.2024 18:04	Linker Address Map	216 KБ

Рисунок 9 - Файл bootstrap.min.js

Создаем в нашем проекте папку static и подключаем её в settings.py. Далее создаем папки css и js и закидываем в них наш bootstrap.

Ваша иерархия должна выглядеть так (рисунок 10):

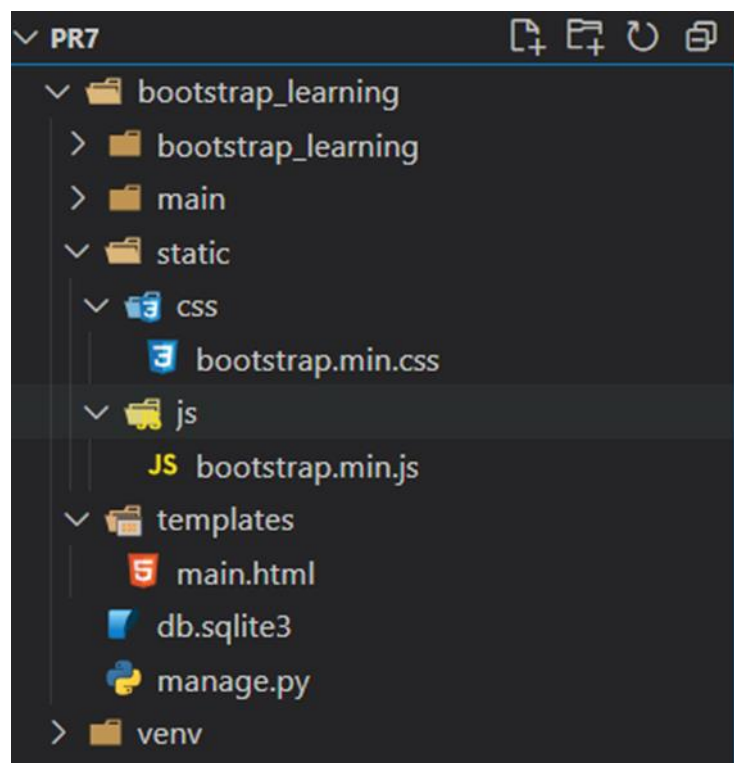
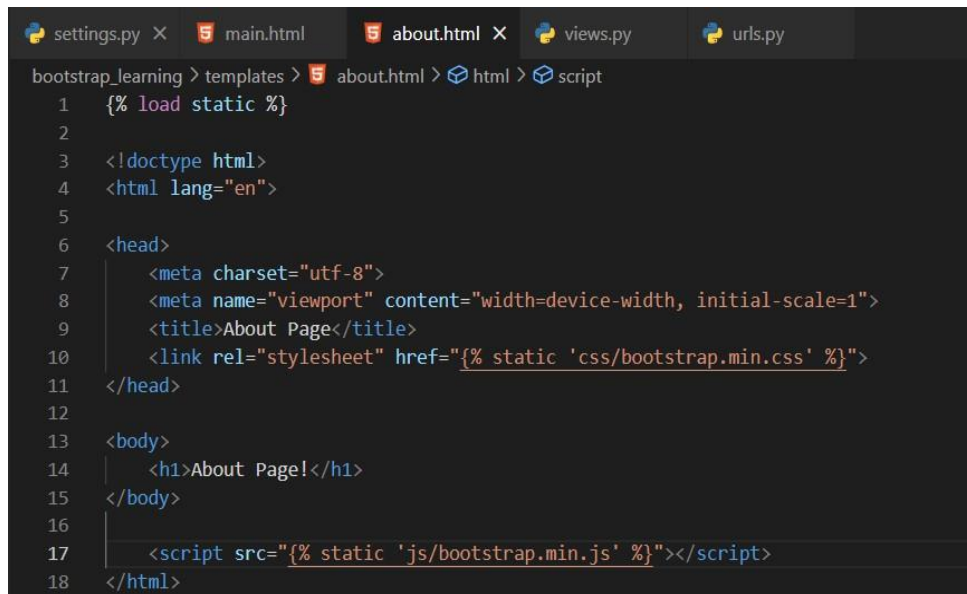


Рисунок 10 - Иерархия проекта

Добавим новую страницу, реализуем ей логику отображения (вьюха). И пропишем такой код для подключения статических файлов (рисунок 11).



```

bootstrap_learning > templates > about.html > html > script
1  {% load static %}
2
3  <!doctype html>
4  <html lang="en">
5
6  <head>
7      <meta charset="utf-8">
8      <meta name="viewport" content="width=device-width, initial-scale=1">
9      <title>About Page</title>
10     <link rel="stylesheet" href="{% static 'css/bootstrap.min.css' %}">
11 </head>
12
13 <body>
14     <h1>About Page!</h1>
15 </body>
16
17     <script src="{% static 'js/bootstrap.min.js' %}"></script>
18 </html>

```

Рисунок 11 - Код страницы about.html

Если Вы все сделали правильно, у Вас будет следующий вид страницы (рисунок 12):

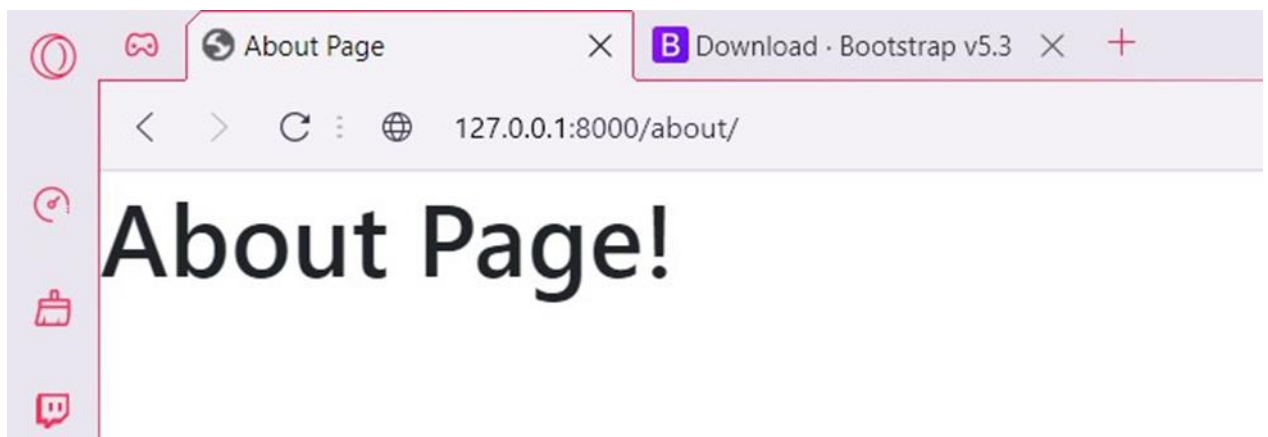


Рисунок 12 - Работающий Bootstrap (статическое подключение)

Как видите, нет никакой разницы в стилях, что тот, что этот способ – рабочие. Но данный способ более эффективный, т.к. все файлы хранятся статически, т.е. внутри нашего проекта.

Bootstrap славится своей простотой и адаптивностью под разные устройства. Краткость классов – главная фишка Bootstrap (рисунок 13).

```
<h3 class="text-center">Заголовок по центру</h3>
```

Рисунок 13 - Класс text-center (выравнивание по центру)

Класс text-center выравнивает текст относительно контейнера по центру страницы (рисунок 14).

```
<div class="container mt-3">
  <h3 class="text-center">Заголовок по центру</h3>
</div>
```

Рисунок 14 - Класс container и mt-3

Класс container – делает обертку для страницы и все элементы будут располагаться внутри данной обертки (рисунок 15).

Заголовок по центру

Рисунок 15 - Класс container и mt-3

Класс mt-x (mt-5, mt-4, mt-3, mt-2, mt-1) – margin top (внешний отступ сверху). Классы создаются на основе карты Sass по умолчанию в диапазоне от .25 rem до 3rem.

```
<p class="lead">Какой-то текст, красиво оформленный</p>
```

Рисунок 16 - Класс lead

Класс lead – делает выделение абзаца (рисунок 17):

```
<div>
  <button type="button" class="btn btn-primary">Primary</button>
  <button type="button" class="btn btn-secondary">Secondary</button>
  <button type="button" class="btn btn-success">Success</button>
  <button type="button" class="btn btn-danger">Danger</button>
  <button type="button" class="btn btn-warning">Warning</button>
  <button type="button" class="btn btn-info">Info</button>
  <button type="button" class="btn btn-light">Light</button>
  <button type="button" class="btn btn-dark">Dark</button>

  <button type="button" class="btn btn-link">Link</button>
</div>
```

Рисунок 17 - Классы кнопок и различные цвета

Классы кнопок. Класс btn – это кнопка, а за ней идет ещё один класс, например, btn-success – означает, что наша кнопка будет зеленого цвета. В bootstrap есть несколько так сказать: «Базовых цветов» (рисунок 18).

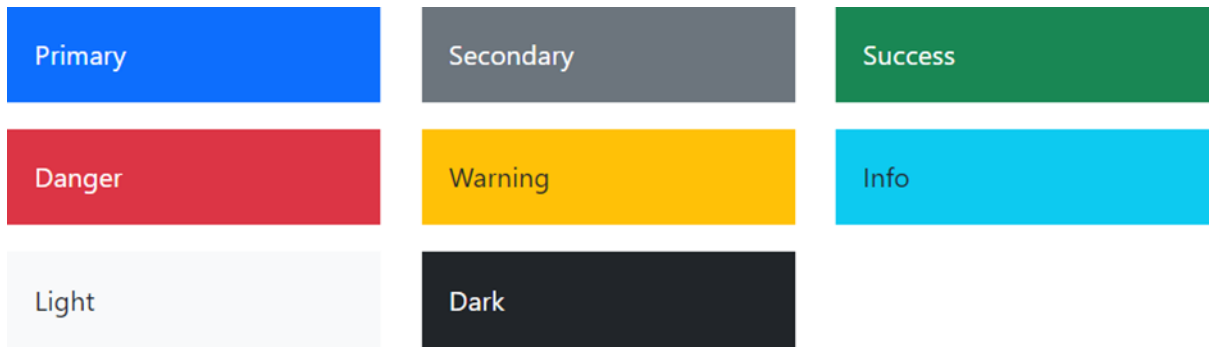


Рисунок 18 - Базовые цвета в Bootstrap

Так выглядят 8 базовых цветов, которые предлагает bootstrap. Обычно, этих цветов более чем хватает, если вы хотите сделать что-то на скорую руку, для более красивого дизайна, вам нужно использовать собственные стили для оформления.

Также, есть ещё кастомные цвета (очень даже красивые) (рисунок 19):

```
<div>
  <button type="button" class="btn btn-outline-primary">Primary</button>
  <button type="button" class="btn btn-outline-secondary">Secondary</button>
  <button type="button" class="btn btn-outline-success">Success</button>
  <button type="button" class="btn btn-outline-danger">Danger</button>
  <button type="button" class="btn btn-outline-warning">Warning</button>
  <button type="button" class="btn btn-outline-info">Info</button>
  <button type="button" class="btn btn-outline-light">Light</button>
  <button type="button" class="btn btn-outline-dark">Dark</button>
  <button type="button" class="btn btn-outline-link">Link</button>
</div>
```

Рисунок 19 - Класс btn-outline

Классы btn-outline – кнопки с hover эффектом (эффект при наведении на кнопку) (рисунок 20):



Рисунок 20 - Отображение кнопок на странице

Вам знакомо такое css-свойство как display-flex? В bootstrap все реализовано за счет колонок (рисунок 21).


```
<h3 class="text-center">Колонки</h3>
<div class="col-12">
  <div class="col bg-success">1</div>
  <div class="col bg-warning">2</div>
  <div class="col bg-info">3</div>
  <div class="col bg-danger">4</div>
  <div class="col bg-success">5</div>
  <div class="col bg-warning">6</div>
  <div class="col bg-info">7</div>
  <div class="col bg-danger">8</div>
  <div class="col bg-success">9</div>
  <div class="col bg-warning">10</div>
  <div class="col bg-info">11</div>
  <div class="col bg-danger">12</div>
</div>
```

Рисунок 21 - Колонки в Bootstrap



Рисунок 22 - Отображение колонок на странице

Если добавить класс `d-flex`, у нас появится как раз-таки та самая колончатая система (рисунки 23-24).

```
<h3 class="text-center">Колонки</h3>
<div class="col-12 d-flex">
  <div class="col bg-success">1</div>
  <div class="col bg-warning">2</div>
  <div class="col bg-info">3</div>
  <div class="col bg-danger">4</div>
  <div class="col bg-success">5</div>
  <div class="col bg-warning">6</div>
  <div class="col bg-info">7</div>
  <div class="col bg-danger">8</div>
  <div class="col bg-success">9</div>
  <div class="col bg-warning">10</div>
  <div class="col bg-info">11</div>
  <div class="col bg-danger">12</div>
</div>
```

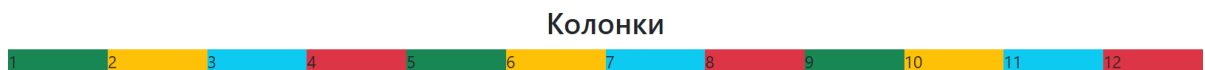
Рисунок 23 - Горизонтальные колонки (добавление класса `d-flex`)

Рисунок 24 - Отображение горизонтальных колонок на странице

Например, перед Вами стоит задача, сделать так, чтобы первая часть содержимого занимала 3 колонки, вторая 6, а третья 3, и в них был какой-то важный текст.

С помощью bootstrap делается это все очень просто (рисунки 25-26).

```
<div class="col-12 d-flex">
  <div class="col-3 bg-success">
    <p class="lead text-center">Тут очень важный текст</p>
  </div>
  <div class="col-6 bg-info">
    <p class="lead text-center">Тут тоже важный текст</p>
  </div>
  <div class="col-3 bg-danger">
    <p class="lead text-center">Не менее важный текст</p>
  </div>
</div>
```

Рисунок 25 - Код для 3 колонок, исходя из поставленной задачи

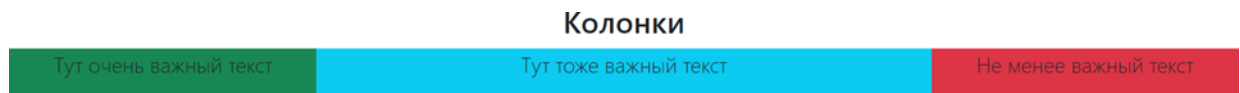


Рисунок 26 - Отображение 3 колонок на странице

О Bootstrap и его возможностях можно говорить очень долго, не говоря уже о выпадающих списках, модальных окнах, слайдеров, пианино и многого другого.

В целом, Bootstrap 5 является мощным инструментом для ускорения и упрощения процесса веб-разработки. Его гибкость и обширные возможности делают его популярным выбором для разработчиков всех уровней.

Пример решения задачи на языке программирования Python

Задача. В ранее созданном проекте **blog_site** и приложении **posts** подключите Bootstrap к шаблону **base.html** с помощью CDN-ссылки. Оформите список публикаций в шаблоне **post_list.html** с использованием компонента **card** и адаптивной сетки Bootstrap (**row**, **col-md-6**). Добавьте кнопку «Читать далее» с классом **btn btn-primary** для каждой публикации.

Решение задачи:

1 Подключение Bootstrap в **base.html**

templates/base.html

```
<!DOCTYPE html>
```

```
<html lang="ru">
```

```
<head>
```

```
  <meta charset="UTF-8">
```



```

<title>Blog Site</title>
<!-- Bootstrap 5 CDN -->
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
<nav class="navbar navbar-dark bg-dark mb-4">
  <div class="container">
    <a class="navbar-brand" href="#">Blog Site</a>
  </div>
</nav>
<div class="container">
  {% block content %}
  {% endblock %}
</div>
<!-- Bootstrap JS -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></sc
ript>

```

```
</body>
```

```
</html>
```

2 Оформление списка публикаций post_list.html

Используем:

grid: row, col-md-6

карточки: card

кнопка: btn btn-primary

templates/posts/post_list.html

```
{% extends 'base.html' %}
```

```
{% block content %}
```

```
<h1 class="mb-4">Публикации</h1>
```

```
<div class="row">
```

```
  {% for post in posts %}
```

```
    <div class="col-md-6 mb-4">
```

```
      <div class="card h-100 shadow-sm">
```

```
        <div class="card-body">
```

```
          <h5 class="card-title">{{ post.title }}</h5>
```

```
          <p class="card-text">
```

```
            {{ post.content|truncatewords:20 }}
```

```
          </p>
```

```
          <a href="{% url 'post_detail' post.id %}" class="btn btn-primary">
```

```
            Читать далее
```

```
          </a>
```

```
        </div>
```

```
    </div>
```

```

        </div>
    {% endfor %}
</div>
{% endblock %}
3 Пример view (если нужно проверить)
views.py
from django.shortcuts import render
from .models import Post
def post_list(request):
    posts = Post.objects.all()
    return render(request, 'posts/post_list.html', {'posts': posts})
4 Пример маршрута
urls.py (app posts)
from django.urls import path
from . import views
urlpatterns = [
    path("", views.post_list, name='post_list'),
]

```